v1.0

ネットワーク基礎再入門

TAKASHI WORKS

ネットワーク基礎再入門

目次

1. 表紙
2. はじめに
3. 序章　ネットワークが生まれるまで
4. 第1章　ネットワークの基礎知識
5. 第2章　物理層 ― ケーブルと電波
6. 第3章　データリンク層とスイッチング
7. 第4章　ネットワーク層とIPアドレス
8. 第5章　ルーティング
9. 第6章　VLAN
10. 第7章　トランスポート層
11. 第8章　アクセス制御とNAT
12. 第9章　冗長化
13. 第10章　DHCPとDNS
14. 第11章　アプリケーション層のサービス
15. 第12章　セキュリティと暗号化
16. 終章　学び直しの総括

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

17. ライセンスについて
18. 奥付
19. 裏表紙

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

はじめに

この本の読み方

この本では、決まった形の要素をくり返し使って情報を整理しています。色や形がそのまま「これは何を書いてある部分か」を表しているので、慣れてしまえば、いちいち説明書きを読まなくても、見た目だけで判断できるようになります。

以下は、本書に登場する6つの要素です。一度だけここで見ておいてください（このうち色つきの箱は4種類で、残り2つ「詳細」「豆知識」は特別な箱を持たない通常の本文です）。

1. 一言要約（黄色の角丸ボックス）

通信の役割を7つの層に分けて整理した設計図が、OSI参照モデルです。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

その節で扱う内容を1文に凝縮したものです。時間がないときは、この黄色い箱だけを拾い読みしていっても、本書全体の大枠はつかめます。

2. スライド風要約（グレーの枠）

なぜ層に分けるか

- 役割ごとに問題を切り分けられる
- どこが壊れているか特定しやすい
- 層ごとに技術を入れ替えられる（例: 無線LANを有線に変えても、上位層の仕組みはそのまま使える）

一言要約をもう一段具体的にした、3つ程度の観点への分解です。ここまで目を通せば、詳細に進まなくても内容の大半は把握できます。

3. 詳細（通常の本文）

スライド風要約で示した内容を、実際の文章として丁寧に説明していく部分です。仕組みや背景をきちんと理解したいときに読む本文で、複数節にわたることもあります。特別な

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

箱の形はなく、この段落のような通常の本文として続きます。

4. 実務メモ（琥珀色の左線）

> 現場で実際に起きる事故や、著者自身が経験した具体的な事例です。

読み飛ばしても、その節の本筋の理解には支障ありません。実務での「あるある」を知っておきたい方向けです。

5. クラウドでは（青緑色の左線）

> クラウドで言うと、Network Load Balancer は L4、Application Load Balancer は L7 を意識した機能を持ちます。

その概念が、実際のクラウドサービスでどう現れるかを紹介します。読者がそのサービスを使っていなくても、事実として読める内容にしています。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

6. 豆知識（節見出しに「豆知識：」と付いた節）

本筋から少し離れた余談や、読者が自宅で簡単に確かめられる小ネタをまとめた節です。見出しに番号が振られた通常の節の一つとして配置していますが、読み飛ばしても、その章の本筋の理解には支障ありません。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

序章　ネットワークが生まれるまで

これから語るのは、コンピュータがまだ1台ずつ孤立して動いていた時代から、今のように世界中がひとつながりになるまでの物語です。実際の規格や年表をそのままなぞったものではなく、事実とはかなり異なっている部分もあります。技術が生まれた順番やきっかけも、分かりやすさを優先して大胆に組み立て直しています。あくまで、全体像をつかむためのイメージとして読んでください。

一台だけでは、何も始まらない

はじまりは、1台のコンピュータでした。計算をし、データを保存し、結果を表示する。それだけなら、他の機械とつながる必要はありません。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode



まだ何ともつながっていない、1台だけのコンピュータ

しかし、隣の部屋にもう1台コンピュータが置かれたとき、事情が変わります。片方で作ったデータを、もう片方でも使いたい。わざわざフロッピーディスクを持って歩くのは、あまりに面倒です。

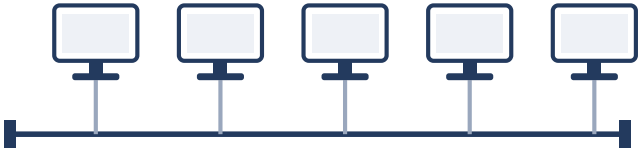
そこで生まれたのが、「つなぐ」という発想でした。2台の機械を同軸ケーブルで結び、電気信号でデータをやり取りする。ネットワークの、いちばん最初の一歩です。

ただし、つないだだけでは終わりません。3台、4台とコンピュータが増えてくると、「今送った信号は、いったいどの機械宛てなのか」を区別する必要が出てきます。ここで、それぞれの機械に固有の名前——アドレス——を割り振るという発想が生まれました。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

一本の線に、みんなでぶら下がる

やがて、1つの部屋に何台ものコンピュータが並ぶようになります。機械同士のつなぎ方はいくつか考えられましたが、広まっていったのは、1本の同軸ケーブルを長く延ばし、そこに全員をぶら下げてしまう方式でした。この方式は、あとからコンピュータを増やしたり減らしたりする際にも、少ない手間で済むという特徴を持っていました。



1本の同軸ケーブルに、全員がぶら下がっている状態

ただし、同軸ケーブルにも限界があります。建物のフロアいっぱいに配線を延ばしていくと、信号はだんだん弱くなり、やがて読み取れなくなってしまうのです。そこで、途中に信号を増幅して送り直す中継器――リピータ――を挟み込み、ケーブルをより長く延ばせるようにしました。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

こうして1本の線にたくさんのコンピュータがぶら下がると、誰かが話し始めた信号は、線を通じて全員に届いてしまいます。ここで生きてくるのが、さきほどのアドレスです。信号には宛先のアドレスが書き込まれており、受け取った側は「これは自分宛てか」を確認し、違っていれば黙って読み捨てる。届くのは全員でも、実際に処理するのは宛先の1台だけ、というわけです。

ただ、それだけでは足りません。誰かが話している最中に別の誰かも話し出せば、信号同士がぶつかってしまいます。そこで考え出されたのが、「話してよい権利」――トークン――を、あらかじめ決めた順番で回していく方式でした。同じ1本の線にみんなでぶら下がったまま、論理的な輪を作ってトークンを回す。これが、トークンバスと呼ばれる方式です。

長い一本の線という弱点

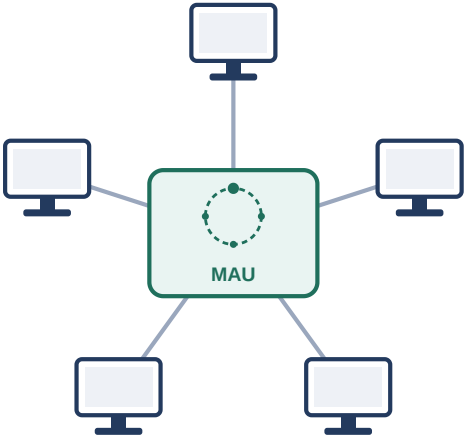
しかし、この1本の共有ケーブルには弱点がありました。フロアいっぱいに延びた同軸ケーブルのどこか1か所が切れただけで、そこから先――ときには全体――が使えなくなっ

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

てしまうのです。しかも、長いケーブルのどこが切れているのかを探し出すのは、簡単な作業ではありません。

　そこで発想が転換されます。1本の長い線をみんなで共有するのではなく、各コンピュータから中央の集線装置まで、1本ずつ専用のケーブルを引く。集線装置の内部で輪とトークンを維持し、どのコンピュータが今つながっているかを把握しておけば、1本のケーブルにトラブルがあっても、影響はその1台だけで済みます。こうして実用化されたのが、「トークンリング」という方式です（この集線装置は、厳密にはMAUと呼ばれますが、詳しくは扱いません）。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode



外から見ると星形の配線だが、集線装置の内部では輪とトークンが維持されている

順番を待たない、という発想

トークンリングは、衝突が起きないという点では優秀な方式でした。ただし、トークンが自分のところに回ってくるまでは話し始められません。誰も話していない状況でも、トークンが一周してくるまでは律儀に待たされてしまいますし、輪を維持するための専用の回路も必要で、コストもかさみます。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

そこで、まったく逆の発想も生まれました。順番を待つのはやめて、あらかじめ回線が空いているかどうかを確認してから話し始める。それでももし誰かと同時に話し出してぶつかってしまったら、そのときはじめて検知して、お互いランダムな時間だけ待ってから送り直せばいい――という考え方です。これが、CSMA/CDと呼ばれる方式でした。

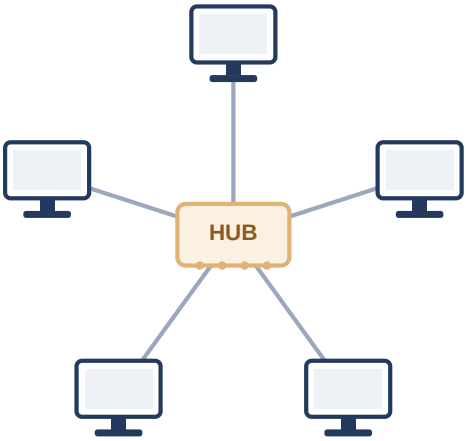


空いているか確認してから送り、ぶつかったら検知して送り直すCSMA/CD

順番を待つ手間がなくなった分、CSMA/CDは速く、そして仕組みがシンプルな分だけ安く作れました。CSMA/CD陣営もやがて、トークンバスと同じ問題――1本の共有ケーブルのどこかが切れると全体が止まってしまう――に行き当たり、同じ答えにたどり着きます。ただし、輪を維持するような複雑な仕組みは必要なく、ただ受け取った電気信号を全員

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

に送り出すだけの、単純な箱で済みました。今でいう「ダムハブ」、リピータハブです。



ダムハブを介して、全員が同じ1つの土俵につながっている状態

CSMA/CDの「譲り合い」のルールは、まさにこの、全員が同じ1つの土俵に上がっている状態でこそ必要とされました。

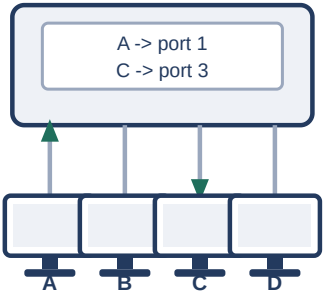
© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

衝突が消えていく

台数がさらに増えると、CSMA/CDの「譲り合い」だけでは追いつかなくなります。ぶつかり合い（コリジョン）があまりに頻繁に起こり、ネットワーク全体の効率が落ちてしまうのです。

この問題に答えを出したのが、L2スイッチでした。それまではダムハブにつながる全員が、衝突の起こりうる範囲――コリジョンドメイン――をまるごと共有していました。スイッチは、受け取った信号の送信元アドレスを覚えておき、「このアドレスはこのポートの先にいる」という対応表を作ります。次に届いた信号は、宛先アドレスをこの対応表と照らし合わせ、該当する1つのポートだけに送り出す。ダムハブのように全員へばらまくのではなく、必要な相手だけに届けられるようになったのです。

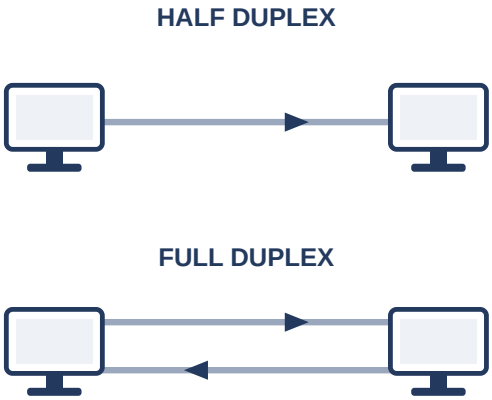
© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode



アドレスとポートの対応表をもとに、必要なポートだけに転送する

この「コリジョンドメインを小さくする」工夫を突き詰めていった結果、最終的には1つのポートに1台だけがつながる形になりました。ちょうど同じころ、配線そのものも同軸ケーブルから、より対線と呼ばれるケーブルに置き換わっていきます（両端のコネクタの形状はRJ45と呼ばれます）。より対線は4対の線を束ねた構造ですが、当時の規格で実際に使われるのはそのうち2対だけでした。1対を送信専用、もう1対を受信専用に割り当てることで、送信と受信が別々の経路を通るようになりました。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode



送受信が同じ経路を取り合う半二重と、別経路で同時にやり取りできる全二重

送信と受信が同時にこなせるこの全二重という仕組みと、1ポート1台という組み合わせが揃ったことで、そもそも衝突が起こる余地そのものがほとんどなくなっていきました。

サーバーという新しい役割

コンピュータ同士がつながると、面白いことが起こります。あるコンピュータは、みんなが使いたいデータやサービスを一手に引き受けるようになっていきました。ファイルを保管する係、印刷を取りまとめる係。周りのコンピュータ

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

は、その係にお願いをする側にまわります。これが、サーバーとクライアントという役割分担のはじまりです。

サーバーは、用意するのも管理するのも手間のかかる、貴重な存在です。だから1台のサーバーに、できるだけ多くの仕事をさせたい。ファイルの受け渡しも、メールのやり取りも、Webページの配信も、全部同じ1台で済ませられれば効率的です。

けれど、1台のサーバーに複数のサービスを同居させると、新しい問題が持ち上がります。届いた通信が、いったいどのサービス宛てなのかを、区別できなければならないのです。ここで生まれたのが、窓口の番号――ポート番号という考え方でした。

組織を越えてつながる

こうして社内・学内といった限られた範囲でのネットワークができあがると、今度は「よそのネットワークともつながりたい」という欲求が生まれます。異なる組織のネットワーク同士を結ぶには、アドレスの体系をもっと大きな規模に広

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

げる必要がありました。ここで登場するのが、IPアドレスと、ネットワーク同士の橋渡し役であるルータです。

ネットワークがネットワークとつながり、そのまたネットワークとつながっていく。この積み重ねの果てに生まれたのが、今わたしたちが当たり前のように使っているインターネットです。

開いた扉と、増えすぎた机

ただし、扉を開けば、招かれざる客も入ってきます。つながる相手が増えるほど、「入れたくない相手を締め出す」という発想が必要になりました。アクセス制御の考え方は、こうして生まれています。

外の世界とのつながりが広がっていく一方で、組織の中でも別の悩みが膨らんでいきました。人が増え、机が増え、機材が増え、フロアが足りなくなる。部署ごとにネットワークを分けたくても、そのたびに専用のスイッチを1台ずつ用意していては、配線もコストもかさむ一方です。「1台の機械の中に、まるで複数の独立したネットワークがあるかのように

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

見せかけられないか」――そんな発想から、次の技術が生まれることになります。

　こうして振り返ると、ネットワークの歴史は、ハードウェアとソフトウェアの技術進歩、コストの最適化、そして複雑さと単純さの間を行き来しながら、絶え間なく形を変えてきた道のりでもあります。ここから先は、この旅の続きを、一章ずつじっくりとたどっていきましょう。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

第1章　ネットワークの基礎知識

1.1　プロトコルとレイヤーという考え方

通信の手順を「プロトコル」として決めておき、役割ごとに独立した「層（レイヤー）」に分けて考える。これが、現代のネットワークを支える設計思想です。

序章では、2台のコンピュータがつながるたびに、新しい「取り決め」が必要になる様子を見てきました。信号が誰宛てかを区別するためのアドレス、衝突を避けるための順番のルール、宛先だけに届けるための対応表――どれも、複数の機械が「こうしましょう」と同じルールに従うことで、はじめて成立する仕組みでした。

このような、通信のために合意されたルールの集合を、プロトコル（protocol）と呼びます。プロトコルは1つとは限りません。実際の通信では、電気信号として0と1を送る仕組み、同じ配線上のどの機械宛てかを区別する仕組み、離れ

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

たネットワークまで届ける仕組み、届いたデータをどのアプリケーションに渡すかを区別する仕組みなど、性質の異なる課題が同時に発生します。これらすべてを1つの巨大なルールブックにまとめてしまうと、ケーブルを光ファイバに変えただけで、ルールブック全体を書き直す羽目になりかねません。

層に分ける利点

- 役割ごとに問題を切り分けられる（「つながらない」原因を、配線の問題か、宛先の指定ミスかで切り分けられる）
- 層ごとに技術を入れ替えられる（無線LANを有線に変えても、その上の仕組みはそのまま使える）
- 標準化された「境界線（インターフェース）」さえ守れば、層の中身は別の会社・別の技術に置き換えてよい

そこで考え出されたのが、通信の課題を性質ごとに独立した「層」に分割し、各層は隣り合う層とだけ決まった約束事でやり取りする、という設計です。手紙にたとえるとわかりやすいかもしれません。手紙の中身（何を伝えるか）と、封筒に書く宛先（誰に届けるか）と、実際に配達する郵便網の仕組み（どうやって物理的に運ぶか）は、まったく別の関心

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

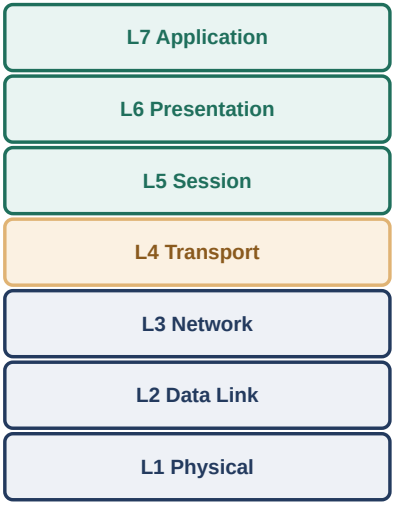
事です。中身をどう書くかは、配達方法が郵便か宅配便かとは無関係に決められますし、逆に配達方法を変えても、手紙の書き方を変える必要はありません。ネットワークの世界でも同じように、「層」ごとに関心事を切り分けることで、一部分だけを独立に変更・改良できるようにしています。

1.2　OSI参照モデル

通信を7つの層に分けて整理した設計図が、OSI参照モデルです。実際のインターネットは、より簡略化した4層のモデルで動いていますが、「どの層の話をしているか」を共有する共通言語として、今でも広く使われています。

この「層に分ける」という考え方を、業界共通の枠組みとして整理したものが、OSI参照モデル（Open Systems Interconnection Reference Model）です。国際標準化機構（ISO）と国際電気標準会議（IEC）が共同で策定した国際規格ISO/IEC 7498-1として定義されており※1、通信の役割を次の7つの層に分けています。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode



OSI参照モデルの7層構造

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

7つの層（下から順に）

- **L1 物理層**: 電気信号や光信号として、0と1をどう表現し、どう送るか（序章で見てきた、同軸ケーブルやより対線の話そのもの。第2章）
- **L2 データリンク層**: 同じ配線・同じ機器の中で、どのMACアドレス宛てかを区別する（第3章）
- **L3 ネットワーク層**: 離れたネットワークまで、IPアドレスをもとに経路を選んで届ける（第4章・第5章）
- **L4 トランスポート層**: どのアプリケーション宛てかを区別し、必要なら信頼性のある配送を保証する（第7章）
- **L5 セッション層**: 通信の開始から終了までの、一連のやり取りをまとめて管理する
- **L6 プレゼンテーション層**: 文字コードや暗号化・圧縮など、データの表現形式を扱う
- **L7 アプリケーション層**: Webページの閲覧やメールの送受信など、利用者が直接触れるアプリケーションの取り決め（第11章）

L1からL4までは、序章から本書がこれまで・これから積み上げていく内容と、ほぼそのまま対応しています。一方、L5〜L7は、実際のインターネットの世界では、OSI参照モデルほどきれいに3つの層へ分かれているわけではありません。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

インターネットの標準を定めるIETF（Internet Engineering Task Force）は、RFC 1122という文書の中で、実務上の通信を「リンク層」「インターネット層」「トランスポート層」「アプリケーション層」という4つの層で捉えるモデルを示しています※2。ここでは、OSIのセッション層・プレゼンテーション層に相当する役割は、独立した層としてではなく、それぞれのアプリケーション（たとえばHTTPSにおけるTLSなど）の中に組み込まれる形で実現されています。

つまり、OSI参照モデルは「実際にこの7層のソフトウェアが機械の中で動いている」という意味の設計図というより、「通信のどの部分の話をしているのか」を関係者どうしで共有するための、共通言語としての性格が強いモデルです。現場で「レイヤー3の障害です」「これはL2の問題ですね」といった言い方をするとき、話し手たちが思い浮かべているのは、まさにこのOSI参照モデルの層番号です。本書でも、この後の章でこの層番号を目印として使っていきます。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

1.3　LANとWAN

限られた範囲・単一の組織が管理するネットワークがLAN、複数の拠点や組織をまたいで広がるネットワークがWANです。

序章で語ってきた物語は、すべて「1つの部屋」「1つの建物」というスケールの中で起きていました。同軸ケーブル、ダムハブ、L2スイッチ――これらは1つの組織が管理する、限られた範囲のネットワークの中の出来事です。このような、単一の建物・キャンパスなど限られた地理的範囲に閉じたネットワークを、LAN（Local Area Network、構内ネットワーク）と呼びます。

これに対して、序章の終盤で登場した「組織を越えてつながりたい」という欲求は、LANの外に踏み出す話でした。地理的に離れた複数の拠点や、異なる組織のネットワーク同士を結ぶ、より広い範囲のネットワークを、WAN（Wide Area Network、広域ネットワーク）と呼びます。自社の複数の拠点を専用線で結ぶ場合も、インターネットを経由して

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

つながる場合も、LANとLANの間を橋渡しする部分はWANの領域です。

LANとWANの違い

- LAN: 1つの組織が自前で管理する、建物・フロア単位の狭い範囲
- WAN: 複数の拠点・複数の組織にまたがる、広い範囲。多くの場合、通信事業者（キャリア）の回線を借りて構築する
- 序章の物語は、ほぼ全編がLANの中の出来事。「組織を越えてつながる」からがWANの領域

序章で登場したIPアドレスとルータは、まさにこのLANとWANの境界で働く道具です。ルータは、自分の組織のLANと、外の世界（他のLAN、あるいはWAN）との橋渡し役を担います。この境界の具体的な仕組みは、第4章で改めて扱います。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

1.4　クライアントサーバーとピアツーピア

サービスを提供する側とお願いする側の役割が固定されているのがクライアントサーバー方式、参加者全員が対等な立場でやり取りするのがピアツーピア（P2P）方式です。

序章では、ファイルを保管する係や印刷を取りまとめる係として、特定の1台のコンピュータがサービスを提供する側に回り、周囲のコンピュータがそれをお願いする側にまわる、という役割分担が生まれる様子を見てきました。サービスを要求する側をクライアント（client）、サービスを提供する側をサーバー（server）と呼び、この役割分担にもとづく通信のあり方を、クライアントサーバーモデルと呼びます。

クライアントサーバーモデルには、管理のしやすさという大きな利点があります。データやサービスが1か所（サーバー）に集約されているため、アクセス権の管理やバックアップ、セキュリティ対策を一元的に行えます。一方で、サーバーは貴重で単一の存在である分、そこが停止したり、想定を

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

超えるアクセスが集中したりすると、サービス全体が影響を受けてしまうという弱点も抱えています。

2つのモデルの対比

- クライアントサーバー: 役割が固定。管理しやすいが、サーバーに負荷や障害が集中しやすい
- ピアツーピア（P2P）: 全参加者が対等。参加者が増えるほど全体の処理能力も増えるが、特定のデータの所在や品質を保証しにくい

これとは異なる発想が、ピアツーピア（Peer-to-Peer、P2P）方式です。P2Pでは、データそのものを特定のサーバーに集約せず、参加者（ピア）全員が対等な立場で、互いにクライアントにもサーバーにもなります。たとえば、ファイル共有プロトコルであるBitTorrentでは、同じファイルをダウンロードしようとしている参加者同士が、自分がすでに持っている部分を互いにアップロードし合うことで、1つの配布元に負荷が集中せずに、多数の参加者へ効率よくファイルを行き渡らせる仕組みになっています※3（参加者同士を引き合わせる案内役として、トラッカーと呼ばれるサーバーが使われることもありますが、ファイルの実データそのものは

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

ピア同士で直接やり取りされます)。参加者が増えるほど、アップロードできる相手も増えるため、理屈のうえでは負荷が分散されていく、という特徴があります。ただし、特定のデータをどのピアが持っているかが常に変動するため、クライアントサーバーモデルほど単純にはデータの所在や可用性を保証できない、という難しさもあります。

1.5　2進数・16進数

コンピュータは電気信号のオン・オフ、つまり2つの状態だけで動いています。この2進数を人間が扱いやすいよう短くまとめて表記するのが、16進数です。

序章で振り返った同軸ケーブルやより対線の話は、結局のところ「電気信号がある状態」と「ない状態」、あるいは「電圧が高い状態」と「低い状態」という、2つの状態の組み合わせで情報をやり取りしていました。この2つの状態を0と1という数字に対応させ、位取り記数法で表現したものが、2進数（binary）です。2進数の1桁を、ビット（bit）と呼びます。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

たとえば2進数の`1101`は、右から順に1の位・2の位・4の位・8の位を表しており、`8+4+0+1=13`、つまり10進数の13にあたります。8個のビットをひとまとめにしたものを、バイト（byte）と呼び、コンピュータが情報をやり取りする基本単位として広く使われています。

ただし、2進数は桁数が多くなりがちで、人間が読み書きするには不便です。そこで使われるのが、16進数（hexadecimal）です。16進数が便利なのは、ちょうど4ビット（2進数4桁）が、16進数のきっちり1桁に対応するという性質があるからです。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

4ビットと16進数1桁の対応

2進数	10進数	16進数	2進数	10進数	16進数
0000	0	0	1000	8	8
0001	1	1	1001	9	9
0010	2	2	1010	10	A
0011	3	3	1011	11	B
0100	4	4	1100	12	C
0101	5	5	1101	13	D
0110	6	6	1110	14	E
0111	7	7	1111	15	F

この性質のおかげで、1バイト（8ビット）はちょうど16進数2桁で表せます。たとえば2進数`11000000`は、上位4ビット`1100`（16進数C）と下位4ビット`0000`（16進数0）に分けて、16進数`C0`と書けます。これは、後の章で登場するサブ

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

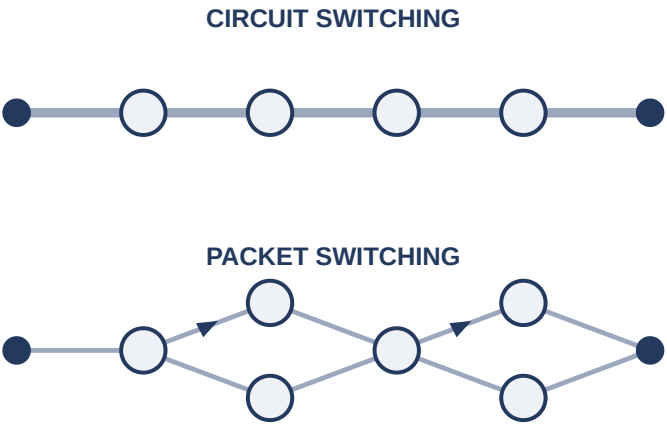
ネットマスクの255（2進数11111111、16進数FF）や、MACアドレス・IPv6アドレスの表記など、ネットワークのさまざまな場面で使われる書き方です。10進数に変換するより、2進数と16進数を行き来するほうが、ビットのまとまりを直感的に把握しやすい、という理由から広く使われています。

1.6　豆知識：回線交換とパケット交換（読み飛ばし可）

序章で語ってきたのは、1つの建物の中で機械同士をつなぐ話でしたが、少し目を広げて、電話網とインターネットの違いにも触れておきましょう。

従来の電話網は、回線交換（circuit switching）という方式で動いていました。電話をかけると、発信者と相手の間に、通話が終わるまで占有し続ける専用の経路が確保されます。この経路は、たとえ会話の合間の沈黙の時間であっても、他の誰かに使われることなく確保されたままです。経路さえ確保できれば、通話品質は安定しますが、経路を使っていない時間も含めて占有してしまうため、大勢が同時に通信しようとすると効率がよくありません。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode



回線交換は経路を占有し続け、パケット交換は小分けにしたデータがそのつど経路を選ぶ

これに対して考え出されたのが、パケット交換（packet switching）という発想です。送りたいデータを小さな単位（パケット）に分割し、それぞれのパケットに宛先の情報を持たせて、ネットワークの中を1つずつ送り出す。各パケットがどの経路を通るかは、そのつど決まってよく、途中の一部が混雑していたり故障していたりしても、別の経路に回り込める、という柔軟性を持っています。

パケット交換という発想は、1960年代にアメリカのランド研究所のポール・バランと、イギリスの国立物理学研究所の

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

ドナルド・デイビスによって、それぞれ独立に考え出されました。バランは、軍事通信網が一部を攻撃されても機能し続けられるようにする、という「生き残れる通信網」の研究の中でこの発想にたどり着き、1964年の研究報告書としてまとめています※4。この報告書は、後にARPANET（インターネットの前身となった実験的ネットワーク）の設計にも影響を与え、今わたしたちが使っているインターネットの土台になりました。序章で見てきた「1本のケーブルが切れると全体が止まる」という共有バスの弱点と同じように、通信網全体をいかに壊れにくくするか、という問いが、ここでも大きな原動力になっていたのです。

参考文献

- ※1: ISO/IEC 7498-1:1994, *Information technology — Open Systems Interconnection — Basic Reference Model: The Basic Model* — https://www.iso.org/standard/20269.html
- ※2: RFC 1122, *Requirements for Internet Hosts -- Communication Layers* — https://www.rfc-editor.org/rfc/rfc1122

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

- ※3: BEP 0003, *The BitTorrent Protocol Specification* — https://www.bittorrent.org/beps/bep_0003.html
- ※4: Paul Baran, *On Distributed Communications: I. Introduction to Distributed Communications Networks*, RAND Corporation, RM-3420-PR, 1964 — https://www.rand.org/pubs/research_memoranda/RM3420.html

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

第2章　物理層 ― ケーブルと電波

2.1　伝送媒体の種類（メタル／光／無線）

信号を運ぶ媒体には、大きく分けて銅線（メタル）、光ファイバ、無線（電波）の3種類があります。

序章では、同軸ケーブルからより対線へと、電気信号を運ぶ銅線（メタルケーブル）の話を中心に見てきました。電気信号は、電圧の高い/低い、あるいは電流の有無で0と1を表現します。銅線は安価で扱いやすい反面、信号が電気である以上、周囲の電磁波（モーターや蛍光灯、他のケーブルなど）の影響を受けやすく、また距離が伸びるほど信号が減衰していく、という制約を抱えています。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

3種類の伝送媒体

- **メタル（銅線）**: 安価で扱いやすいが、電磁ノイズの影響を受けやすく、距離の制約も大きい
- **光ファイバ**: 電気ではなく光（光のパルス）で信号を送るため、電磁ノイズの影響を受けず、非常に長い距離まで届く。ただしケーブル・機器のコストは高め
- **無線（電波）**: ケーブルそのものが不要。設置の自由度は高いが、電波は空間を共有するため、他の電波との干渉や、距離・障害物による減衰を受けやすい

これに対して、光ファイバは電気の代わりに光（レーザーやLEDによる光のパルス）で信号を送ります。光は電磁ノイズの影響を受けないため、銅線よりはるかに長い距離を安定して届けられます。ただし、光を送受信するための機器やケーブルの加工には相応のコストがかかります。

無線（電波）は、ケーブルという物理的な制約そのものから解放される伝送媒体です。設置の自由度は高い一方、電波という「空間」を複数の機器で共有することになるため、他の電波との干渉や、障害物・距離による減衰の影響を強く受けます。この後の節では、メタルケーブル（2.2）、光ファイ

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

バ（2.3）、無線LAN（2.5）について、それぞれもう少し詳しく見ていきます。

2.2　LANケーブルのカテゴリと選び方（Cat5e/6/6A、10Gを確実に飛ばすには）

より対線のケーブルには「カテゴリ」という規格があり、数字が大きいほど、より高速な通信に、より確実に対応できます。

序章で見てきたより対線（RJ45で接続するケーブル）には、「カテゴリ（Cat）」という規格による格付けがあります。カテゴリの数字が大きいほど、対応する周波数帯域が広く、より高速な通信を安定して通せます。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

主なカテゴリの違い

カテゴリ	対応帯域	1Gbps	10Gbps
Cat5e	100MHz	100m	非対応
Cat6	250MHz	100m	55m まで※1
Cat6A	500MHz	100m	100m※1

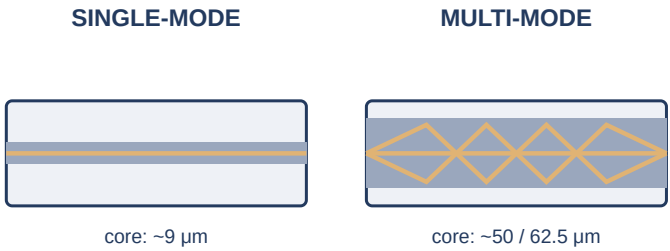
ここで実務上とくに注意したいのが、Cat6 と Cat6A の違いです。Cat6 は10Gbps の通信（10GBASE-T）にも対応してはいますが、その対応距離はケーブル配線の規格上限とされる100m よりずっと短い、55m 程度に制限されます※1。オフィスのフロア配線のように、決まった長さのケーブルを壁の中に通してしまっている環境で「あとから10G に増速したい」と思っても、Cat6 配線のままでは100m の距離を確実に10Gbps で通せる保証がありません。10Gbps を100m の距離で確実に飛ばしたいなら、帯域を倍の500MHz に強化したCat6A を選んでおくのが安全です。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

2.3　光ファイバの基礎（シングル/マルチモード、コネクタ、距離）

光ファイバには、芯（コア）の太さによって、シングルモードとマルチモードの2種類があり、それぞれ得意な距離が異なります。

光ファイバは、中心にある芯（コア）を光が通り抜けることで信号を伝えます。このコアの太さによって、光ファイバは大きく2種類に分かれます※2。



シングルモードは光が1本の経路のみを、マルチモードは複数の経路を通る

シングルモードファイバは、コアの直径が9マイクロメートル程度と非常に細く、光がほぼ1本の経路（モード）しか

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

通れません。経路が1本に絞られるため、複数の経路を通った光が到着時刻のずれを起こす現象（モード分散）が起きず、非常に長い距離まで信号を届けられます。長距離の基幹回線や、通信事業者の回線などで使われます。

マルチモードファイバは、コアの直径が50マイクロメートルや62.5マイクロメートルと太く、光が複数の経路で反射しながら進みます。経路ごとに光の到着時刻がわずかにずれるモード分散が起こるため、シングルモードほど長距離には向きませんが、発光素子（LED等）が安価に作れるため、機器・ケーブルのコストを抑えられます。建物内やデータセンター内など、比較的短い距離の配線でよく使われます。

コネクタにはいくつかの種類がありますが、現在よく使われるのはLCコネクタ（SCの半分程度と小型）と、やや大きめのSCコネクタ（押し込んで固定するタイプ）です。どちらもシングルモード・マルチモードの両方に対応しています※2。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

2.4　ネゴシエーションという仕組み（1Gと10Gの違い）

接続した機器同士が、自動的にお互いの対応速度や通信方式をすり合わせる仕組みが、オートネゴシエーションです。

序章の終盤で、より対線が1対を送信専用・1対を受信専用に使い分けることで全二重が実現した、という話をしました。ただし、接続する2台の機器が「本当に同じ速度・同じ全二重で通信してよいか」を、あらかじめ人間が設定しておく必要があるとしたら、大変面倒です。

この調整を自動で行う仕組みが、オートネゴシエーション（自動ネゴシエーション）です。IEEE 802.3規格のClause 28として定義されており、ケーブルを挿した瞬間、両端の機器がFLP（Fast Link Pulse）と呼ばれる特別な信号のやり取りを通じて、自分が対応できる速度・全二重/半二重の組み合わせを相手に伝え合います。そのうえで、双方が対応できる中で最も高性能な組み合わせを自動的に選びます※3。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

オートネゴシエーションの実務上の注意点

- 片方の機器だけを手動で特定の速度・全二重/半二重の設定に固定すると、もう片方は自動判定に失敗し、速度や全二重/半二重の食い違い（デュプレックスミスマッチ）が起きることがある
- 症状は「つながってはいるが、やたら遅い」「エラーが増える」といった分かりにくい形で現れやすい
- 基本的には両端とも自動（オート）のままにしておくのが無難

10Mbps・100Mbpsの時代は、オートネゴシエーションはあくまで任意の機能で、両端を手動で固定することもできました（だからこそ、片方だけ固定してしまうデュプレックスミスマッチ事故が起きたのです）。これに対し、1Gbps以上の規格（1000BASE-Tや10GBASE-Tなど）では、そもそもオートネゴシエーションによる速度合わせが必須の仕組みとして組み込まれており、手動で固定するという選択肢自体が用意されていません。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

> 実務では、スイッチのCLI上に「速度1000Mbps・全二重」のような設定コマンドが用意されていることがあります。これは、ネゴシエーションの仕組みそのものを止めているのではなく、申告する候補を1つに絞り込んでいるだけです※4。ごく一部の古い機器では、この設定によって本当にネゴシエーションが無効化され、規格から外れた挙動になることもあります。

2.5　無線LAN規格の変遷（Wi-Fi 4→7、理論値の推移）

無線LANの規格は「Wi-Fi 4」のように世代番号で呼ばれるようになり、世代を追うごとに理論上の最大速度が大きく伸びてきました。

無線LANの規格は、本来IEEE 802.11の後ろに付くアルファベット（802.11n、802.11acなど）で区別されていましたが、一般の利用者には分かりにくいため、業界団体のWi-Fi Allianceが「Wi-Fi 4」のような世代番号による呼び方を導

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

入しました。この番号は802.11n以降にさかのぼって付け直されたものです※5。

主な世代と特徴

- **Wi-Fi 4（802.11n、2009年）**: 2.4GHz帯・5GHz帯の両方に対応し、理論値で最大600Mbps
- **Wi-Fi 5（802.11ac、2013年）**: 5GHz帯に特化し、理論値で最大約6.9Gbps
- **Wi-Fi 6（802.11ax、2021年）**: OFDMA（複数端末への効率的な帯域の割り当て）やTWT（省電力機能）により、理論値で最大約9.6Gbpsに加え、混雑した環境での実効性能を重視
- **Wi-Fi 6E**: Wi-Fi 6の技術をそのままに、対応周波数帯を6GHz帯まで拡張
- **Wi-Fi 7（802.11be、2024年）**: 複数の周波数帯を同時に束ねて使うマルチリンク動作により、理論値で最大約46Gbps

ここで挙げた理論値は、いずれも実験室環境での最大値です※5。実際の通信速度は、電波の届く距離、障害物、同時に接続している端末の台数、周囲の電波干渉など、さまざまな要因で理論値より大きく下がります。世代を追うごとに理

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

論値が伸びているのは事実ですが、「Wi-Fi 6だから必ず速い」とは限らない、という点には注意が必要です。

2.6　PoE（Power over Ethernet）

PoE（Power over Ethernet）は、データ通信に使うのと同じLANケーブルを使って、電力も一緒に送ってしまう仕組みです。

より対線には4対の線があり、通信規格によっては2対しか使わない、という話を序章でしました。PoEは、まさにこの「空いている2対」に直流電力を流すか、あるいはデータで使っている対に電力を重畳させる（信号と電力を同じ線に同時に乗せる）ことで、通信と同時に電力を送り届けています。電力を送り出す側（スイッチやPoE対応の電源装置）をPSE（Power Sourcing Equipment）、電力を受け取る側（IP電話や無線LANのアクセスポイント、防犯カメラなど）をPD（Powered Device）と呼びます。PoEを使えば、PDの近くにコンセントがなくても、LANケーブル1本で通信と給電を同時にまかなえます。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

PoE規格と電力

規格	通称	PSE側の供給電力	PD側の受電電力
IEEE 802.3af	PoE	約15.4W	約12.95W
IEEE 802.3at	PoE+	約30W	約25.5W
IEEE 802.3bt Type 3	PoE++	約60W	約51W
IEEE 802.3bt Type 4	PoE++	約90W	約71.3W

世代が進むほど、供給できる電力は大きくなっています※6。PSE側の供給電力とPD側の受電電力に差があるのは、ケーブルの途中で電力の一部が失われることをあらかじめ見込んだ規格になっているためです。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

スイッチには、PoEポート1つあたりの上限とは別に、スイッチ全体としての「電力予算」が決められています。たとえば1台のスイッチの総電力予算が370Wだった場合、PoE++対応の高出力カメラを何台も接続していくと、各ポートの上限（60W等）以内に収まっていても、スイッチ全体の予算を使い切ってしまうことがあります。この場合、あるポートに機器をつないでも電源が入らなかったり、既存の機器が突然給電を止められたりする、という分かりにくい現象が起こります。PoE機器を増設する際は、ポート単位の上限だけでなく、スイッチ全体の電力予算にも余裕があるかを確認する必要があります。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

> クラウドを利用しているとき、わたしたちがどんな種類のケーブルや光ファイバの上で通信しているかを意識することは、ほとんどありません。物理層の管理は、データセンターを運用するクラウド事業者側の責任範囲だからです。それでも物理層の影響がまったく見えなくなるわけではなく、たとえば「利用者に近いリージョンを選ぶと通信の遅延が小さくなる」「仮想ネットワークのゲートウェイに帯域の上限プランがある」といった形で、物理的な距離や回線容量の制約が、サービスの選択肢として間接的に姿を現します。

参考文献

- ※1: *Ethernet Cable Standards Explained: TIA-568, Cable Categories, and the Truth About CAT6E* — Vertical Cable — https://verticalcable.com/blog/ethernet-cable-standards-explained

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

- ※2: *Ethernet physical layer* ― Wikipedia ― https://en.wikipedia.org/wiki/Ethernet_physical_layer
- ※3: *Autonegotiation* ― Wikipedia ― https://en.wikipedia.org/wiki/Autonegotiation
- ※4: Cisco, *Configure and Verify Ethernet 10/100/1000Mb Half/Full Duplex Auto-Negotiation* ― https://www.cisco.com/c/en/us/support/docs/lan-switching/ethernet/10561-3.html
- ※5: *The Evolution of Wi-Fi Technology and Standards* ― IEEE Standards Association ― https://standards.ieee.org/beyond-standards/the-evolution-of-wi-fi-technology-and-standards/
- ※6: *Power over Ethernet Standards (802.3af, 802.3at, & 802.3bt)* ― NVT Phybridge ― https://www.nvtphybridge.com/power-over-ethernet-standards/

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

第3章　データリンク層とスイッチング

3.1　MACアドレスとフレーム

すべてのネットワーク機器には、世界で重複しない48ビットの識別番号「MACアドレス」が割り当てられており、通信の最小単位である「フレーム」の中に、宛先・送信元として書き込まれます。

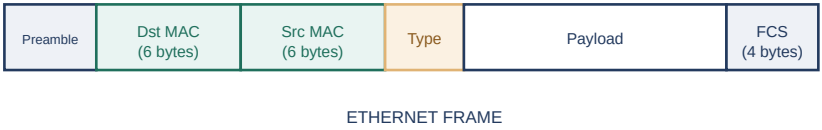
序章では、複数の機械を区別するために「アドレス」という発想が生まれる様子を見てきました。データリンク層（L2）で使われるこのアドレスを、正式にはMACアドレス（Media Access Control address）と呼びます。MACアドレスは48ビット（6バイト）の数値で、第1章で扱った16進数を使い、`00:1A:2B:3C:4D:5E`のように、2桁の16進数を6組並べて表記します。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

MACアドレスの内訳

- 前半24ビット（3バイト）：OUI（Organizationally Unique Identifier）。IEEE Registration Authorityが、製造元の企業ごとに割り当てる識別子※1
- 後半24ビット（3バイト）：製造元が自社の製品ごとに割り振る、重複しないシリアル番号
- 前半（誰が作ったか）と後半（どの個体か）の組み合わせにより、世界中のあらゆる機器のMACアドレスが重複しないようになっている

このMACアドレスを使って、データリンク層でやり取りされるデータの単位を、フレーム（frame）と呼びます。序章で「信号」と呼んでいたものの正体は、実際にはこのフレームという構造を持ったデータのかたまりです。



イーサネットフレームの基本構造

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

フレームの先頭には、受信側が同期を取るためのプリアンブルが置かれ、続いて宛先MACアドレス・送信元MACアドレスが並びます。序章で見た「宛先が自分あてかどうかを確認して、違えば黙って捨てる」という仕組みは、まさにこの宛先MACアドレスを見て行われています。そのすぐ後ろにあるタイプ（EtherType）フィールドは、続くペイロードがどの上位プロトコルのデータなのかを示す2バイトの値で、たとえば`0x0800`はIPv4、`0x0806`はARPを表します。受信側はこの値を見て、フレームの中身をどのプロトコルの処理へ渡せばよいかを判断します。その後にペイロード（実際に運びたいデータ）が続き、最後にFCS（Frame Check Sequence）という誤り検出符号が付きます。受信側は、届いたフレームからFCSを計算し直し、値が一致するかどうかで、通信中にデータが壊れていないかを確認します※2。

3.2　スイッチ／ハブの役割

ハブは受け取った信号を全ポートにそのまま流すだけの単純な機械、スイッチは宛先MACアドレスを見て必要なポートだけに転送する、賢い機械です。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

序章で見てきたとおり、ダムハブ（リピータハブ）は、受け取った電気信号を、つながっている全員にそのまま送り出すだけの単純な機械でした。これはOSI参照モデルでいうL1（物理層）の働きにとどまり、フレームの中身を一切見ていません。

これに対してL2スイッチは、届いたフレームの宛先MACアドレスを読み取り、そのMACアドレスがどのポートの先にいるかを記録した対応表（MACアドレステーブル）と照らし合わせて、該当する1つのポートだけにフレームを転送します。この対応表は、各ポートに届いたフレームの送信元MACアドレスを見るたびに、「このアドレスはこのポートの先にいる」と自動的に学習していく仕組みになっています。ハブがL1の単純な繰り返し装置であるのに対し、スイッチはL2の情報（MACアドレス）をもとに判断する、賢い機械だといえます。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

3.3　コリジョンドメインとブロードキャストドメイン

信号がぶつかり合う可能性のある範囲がコリジョンドメイン、ブロードキャスト（宛先を指定しない一斉送信）が届く範囲がブロードキャストドメインです。

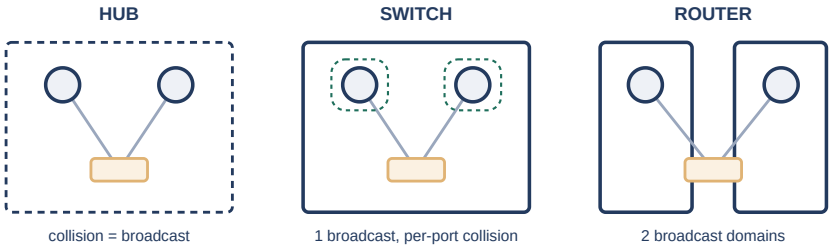
序章で「コリジョンドメイン」という言葉が先出しされていたのを覚えているでしょうか。ここで、この言葉の正式な意味を整理しておきましょう。

コリジョンドメインとは、複数の機器が信号を同時に送り出したときに、衝突が起こりうる範囲のことです。ダムハブにつながる機器は全員が信号を共有するため、ハブにつながる範囲全体が1つのコリジョンドメインになります。一方、L2スイッチは、ポートごとに信号を電気的に分離するため、スイッチの各ポートがそれぞれ独立したコリジョンドメインになります。

これとは別に、ブロードキャストドメインという範囲もあります。ブロードキャストとは、宛先を特定の1台に絞らず、その範囲にいる全員に一斉送信する通信方式のことです。ブ

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

ロードキャストドメインとは、あるブロードキャストが実際に届く範囲を指します。ここで重要なのは、L2スイッチはコリジョンドメインをポートごとに分割しても、ブロードキャストドメインまでは分割しない、という点です。スイッチにつながる機器はすべて、依然として1つのブロードキャストドメインに属しています。ブロードキャストドメインを分割できるのは、L3（ネットワーク層）で動作するルータだけです※3。



ハブ・スイッチ・ルータで、コリジョンドメインとブロードキャストドメインの分かれ方が異なる

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

3つの機器の違い

- ハブ: コリジョンドメイン＝ブロードキャストドメイン。両方とも1つにまとまったまま
- スイッチ: コリジョンドメインはポートごとに分割される。ブロードキャストドメインは1つのまま
- ルータ: ブロードキャストドメインも分割される。ルータの各インターフェースが、別々のブロードキャストドメインの境界になる

3.4　ダムハブ→CSMA/CD→L2スイッチ→全二重化

序章の技術変遷は、「コリジョンドメインをどう扱うか」という1本の軸で説明できます。

序章では、同軸ケーブルによるバス配線から始まり、トークンバス・トークンリング、そしてCSMA/CDという「順番を待たない」発想を経て、ダムハブとL2スイッチ、より対線による全二重化へと至る流れを、物語として追いかけてきま

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

した。この章で登場した用語を使って、その流れを技術的に整理し直すと、次のようになります。

用語で振り返る序章の流れ

- ダムハブ: L1（物理層）で電気信号をそのまま繰り返す装置。全ポートが1つのコリジョンドメイン
- CSMA/CD: L2（データリンク層）で、コリジョンドメインを共有する複数の機器が衝突を検知・回避するための取り決め（IEEE 802.3）
- L2スイッチ: MACアドレステーブルにもとづき、ポートごとにコリジョンドメインを分割する装置
- 全二重化: より対線の送信専用対・受信専用対を活かし、1ポート1台の構成と組み合わさることで、コリジョンドメインという概念そのものが実質的に不要になった状態

つまり、序章で描かれた技術の変遷は、「コリジョンドメインをどう扱うか」という1本の軸で説明できます。最初はコリジョンドメインをみんなで共有し、譲り合いのルール（CSMA/CD）でしのいでいた状態から、スイッチによってコリジョンドメインを1ポート単位まで小さくし、最終的には全二重化によってコリジョンドメインという概念自体が意

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

味を持たなくなる状態へと進化してきた、というのが、この章までの技術的な結論です。

3.5　なぜループ対策が必要か（STPの目的と、STPに頼らない設計）

スイッチを冗長化のために複数のケーブルでループ状につなぐと、ブロードキャストフレームが永遠に回り続ける「ブロードキャストストーム」が起こります。これを防ぐのが、STP（スパニングツリープロトコル）です。

第9章で改めて扱いますが、ネットワークの信頼性を高めるために、スイッチ同士を複数のケーブルで冗長に接続したくなる場面があります。ところが、L2の世界でスイッチ同士を単純にループ状につなぐと、深刻な問題が起こります。

ブロードキャストフレームは、受け取ったスイッチが、受け取ったポート以外のすべてのポートへ転送する仕組みです。ループ状の配線があると、あるスイッチが転送したブロードキャストフレームが、別の経路を回って同じスイッチに戻ってきてしまい、そこでまた全ポートへ転送される――これが際限なく繰り返されます。この現象をブロードキャスト

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

ストームと呼び、ネットワーク帯域を瞬く間に食いつぶし、実質的に通信不能な状態に陥ります。

この問題を防ぐために考え出されたのが、STP（Spanning Tree Protocol、スパニングツリープロトコル）です。IEEE 802.1Dとして標準化されており※4、スイッチ同士が情報をやり取りして、ネットワーク全体の中から「木構造（ツリー）」、つまりループのない経路だけを選び出します。ループを作る余分な経路にあたるポートは、通常時は論理的に閉じて（ブロッキング状態にして）通信に使わないようにしつつ、選ばれた経路に障害が起きたときには、閉じていたポートを開いて経路を自動的に切り替えます。冗長性（複数の物理的な経路）を保ちながら、論理的にはループのない1本の木構造だけを使う、という発想です。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

STPの弱点と、それに頼らない設計

- STPは、障害発生時にブロッキングされていたポートを開くまでに、一定の切り替え時間がかかる
- この切り替え時間の間は通信が途切れるため、より高速な切り替えを求める環境では不向きな場合がある
- そのため近年は、L2のループとSTPに頼るのではなく、後述する冗長化技術（LAGなど、第9章）や、そもそもL2のループが生じない構成（L3による設計）を優先する考え方も広がっている

STPそのものの詳しい設定方法には立ち入りませんが、「なぜループを避ける必要があるのか」「なぜ冗長化とループ回避を両立させる仕組みが必要になるのか」という背景は、第9章で扱う冗長化技術を理解するうえでの土台になります。

参考文献

- ※1: *IEEE Registration Authority (MA-L / OUI)* — IEEE Standards Association — https://standards.ieee.org/products-programs/regauth/oui/

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

- ※2: *Ethernet IEEE 802.3 Frame Format / Structure* — Electronics Notes — https://www.electronics-notes.com/articles/connectivity/ethernet-ieee-802-3/data-frames-structure-format.php
- ※3: *Collision Domain vs. Broadcast Domain* — Baeldung on Computer Science — https://www.baeldung.com/cs/collision-broadcast-domains
- ※4: *What is IEEE 802.1D? Spanning Tree Protocol (STP) & Bridging Explained* — Comms InfoZone — https://www.comms-express.com/infozone/article/802-1d/

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

第4章　ネットワーク層とIPアドレス

4.1　IPアドレスの基礎

IPアドレスは、ネットワーク層（L3）で使われる、ネットワークをまたいで機器を識別するための32ビットの番号です。

序章の終盤で、組織を越えてネットワーク同士をつなぐために、IPアドレスとルータが登場する様子を見てきました。IPアドレス（IPv4アドレス）は32ビットの数値で、8ビットずつ4つに区切り、それぞれを10進数にして.（ドット）でつないだ形（例: `192.168.1.10`）で表記します。普段目にする`192.168.1.10`という表記は、あくまで人間が読みやすくするための10進数表現で、実体は32個の0と1が並んだ、ただの2進数の数値です。第1章1.5節で見た2進数`11000000`（16進数`C0`）は、実は10進数にすると192、つまりこのIPアドレスの先頭8ビットとまったく同じ並びです。

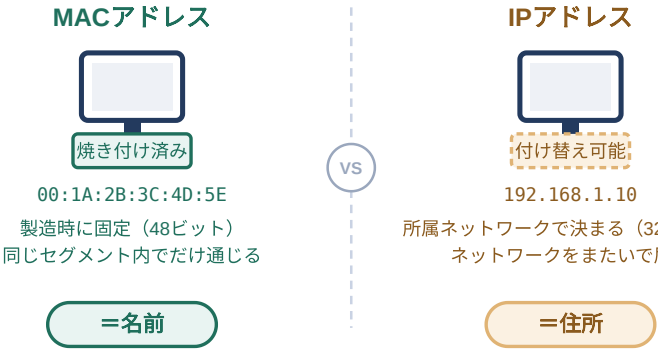
© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode



IPv4アドレスの正体は、8ビットずつ4つに区切られた32ビットの数値

MACアドレスとの違いを整理しておきましょう。MACアドレスは、機器が製造された時点で焼き付けられる固定の識別子で、同じ配線・同じセグメントの中でしか意味を持ちません。これに対してIPアドレスは、機器がどのネットワークに所属しているかに応じて割り当てられる、住所のような階層的な番号です。引っ越しをすれば住所（IPアドレス）は変わりますが、個人を示す名前（MACアドレス相当のもの）は変わらない、というイメージに近いかもしれません。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode



MACアドレスは「名前」、IPアドレスは「住所」に近いイメージ

プライベートIPアドレスとグローバルIPアドレス

- インターネット上で直接やり取りできる、世界で重複しないIPアドレスを、グローバルIPアドレスと呼ぶ
- 一方、組織内など限られた範囲だけで使う前提の、インターネットには直接ルーティングされないIPアドレスの範囲が、あらかじめ予約されている（プライベートIPアドレス）
- RFC 1918として標準化されており、`10.0.0.0`〜`10.255.255.255`・`172.16.0.0`〜`172.31.255.255`・`192.168.0.0`〜`192.168.255.255`の3つの範囲が該当する※1

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

社内LANでよく見かける`192.168.x.x`や`10.x.x.x`といったアドレスは、この予約されたプライベートIPアドレスの範囲を使っている、というわけです。

ここで紹介した「グローバルIPアドレス」には、「パブリックIPアドレス」という、もう1つの呼び方もあります。「世界で重複しない」という性質に注目すればグローバル、「プライベート（私的）」の対義語としての「パブリック（公開）」に注目すればパブリック、というように、どちらの呼び方にもそれぞれ理由があります。クラウドの管理画面（Azure、AWSなど）では「パブリックIPアドレス」という表記が一般的なので、実務ではこちらの呼び方に出会う機会も多いはずです。本書では、以降もグローバルIPアドレスという呼び方で統一しますが、どちらで呼ばれても戸惑わないよう、2つとも覚えておくとよいでしょう。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

4.2　サブネットマスクとサブネッティング

サブネットマスクは、32ビットのIPアドレスのうち、どこまでが「ネットワークを表す部分」で、どこからが「そのネットワーク内の機器を区別する部分」かを示す目印です。

IPアドレスの32ビットは、そのまま全体が1台の機器を指すわけではありません。前半の一部が「どのネットワークに属しているか」を表すネットワーク部、残りが「そのネットワークの中の何番目の機器か」を表すホスト部として使われます。この境界線がどこにあるかを示すのが、サブネットマスクです。

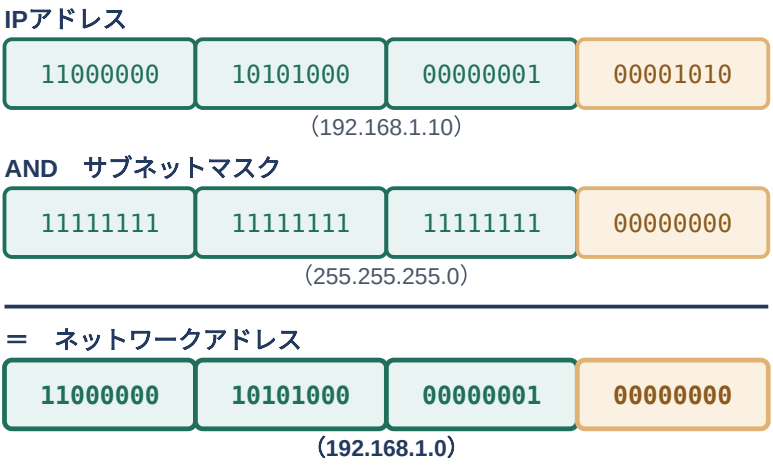
サブネットマスクの正体は、IPアドレスと同じ32ビットの数値です。`255.255.255.0`のように、IPアドレスと並べて設定します。第1章1.5節で「後の章で登場する」と予告した`255`（2進数`11111111`、16進数`FF`）は、まさにこのサブネットマスクに登場する値です。2進数で見ると`11111111.11111111.11111111.00000000`のように、1が並ぶ

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

部分と0が並ぶ部分からできており、1の範囲がネットワーク部、0の範囲がホスト部に対応します。

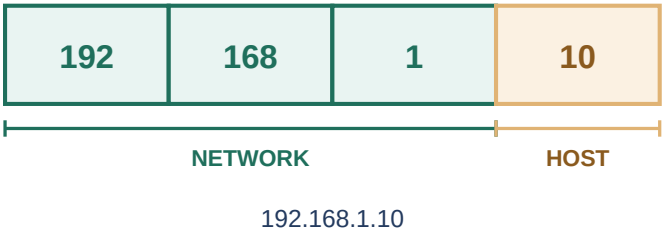
「マスク」という名前は、この数値の使い方に由来します。IPアドレスとサブネットマスクを、ビットごとにAND演算（両方とも1のときだけ結果が1になる、掛け算のような演算）すると、マスクが0になっているホスト部のビットは強制的に0になり（覆い隠され＝マスクされ）、マスクが1になっているネットワーク部のビットだけがそのまま残ります。この演算によって、IPアドレスからネットワーク部だけを取り出せる、というわけです。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode



IPアドレスとサブネットマスクをAND演算すると、ネットワークアドレスが得られる

このAND演算の結果を、普段よく目にする10進数表記で描き直すと、次のようになります。



IPアドレスは、ネットワーク部とホスト部に分かれる

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

同じネットワーク部を持つ機器同士は、間にルータを挟まずに直接やり取りできる「同じネットワークの中」にいる、とみなされます。1つの大きなネットワークを、部署やフロアといった単位でいくつかの小さなネットワークに分割したいとき、ホスト部の一部をさらにネットワーク部として使うことで、より細かいネットワークに区切ることができます。この分割作業を、サブネッティングと呼びます。細かく分割すればするほど、1つのネットワークに含まれる機器の台数は減っていきます。これは、第3章で見たブロードキャストドメインの範囲を、意図的に小さく保つ効果にもつながります。

ここまでは、サブネットマスクを自分たちの都合で自由に決められる前提で話を進めてきました。しかし、インターネットの黎明期には、ネットワーク部の長さは組織ごとに自由に決められるものではなく、「クラス」と呼ばれる3種類の固定長のどれかに、IPアドレスの先頭数ビットのパターンに応じて機械的に当てはめられていました※2。この仕組みを、クラスフルアドレッシングと呼びます。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

クラスA〜Eとデフォルトマスク

クラス	先頭オクテット	デフォルトマスク	ホスト数の上限
A	1〜126	255.0.0.0	約1677万
B	128〜191	255.255.0.0	65,534
C	192〜223	255.255.255.0	254
D	224〜239	（マルチキャスト専用）	—
E	240〜255	（予約／未使用）	—

先頭オクテットを2進数で見比べると、クラスを分ける仕組みの設計意図が見えてきます。クラスAは`0xxxxxxx`、クラスBは`10xxxxxx`、クラスCは`110xxxxx`、クラスDは`1110xxxx`、クラスEは`1111xxxx`と、先頭に並ぶ1の個数がクラスごとに1つずつ増えていきます。最初に0が現れるビットの位置も、クラスAは1桁目、クラスBは2桁目、クラスCは

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

3桁目、クラスDは4桁目と、見事に1桁ずつ後ろにずれていきます（クラスEだけは目印ビットの4ビットすべてが1で、0が現れません）。この「目印ビット」が1ビット長くなるたびに、残りの可変部分は1ビットずつ短くなるため、各クラスが使えるアドレス空間は、クラスAから順に残りの空間の半分ずつに切り出されていきます。クラスDとクラスEは、ともに4ビットの目印（`1110`と`1111`）を使う同じ長さなので、2つで最後に残った領域をちょうど半分に分け合う形になっています。クラスDが専用に予約されているマルチキャストとは、1台の送信元から、特定の複数の相手へ同時に同じデータを届ける通信方式のことです。

なお、クラスAの先頭オクテットは`0xxxxxxx`なので原理上は0〜127ですが、`0`は「このネットワーク自身」を指す特殊な値、`127`はループバック用に予約されているため、表の範囲は実際に割り当て可能な1〜126としています。

社内LANで見かける`192.168.x.x`が、常に先頭24ビットまでがネットワーク部（クラスC相当）であるのは、このクラスフルアドレッシングの名残です。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

しかし、この仕組みには大きな弱点がありました。たとえば1,000台の機器を持つ組織があったとします。クラスBを1つ割り当てれば65,534台分の余裕があり無駄が多すぎますが、クラスCを1つだけ割り当てると254台分しかなく足りません。結局、複数のクラスCをかき集めて使う、というようなことが起こり、無駄なアドレス消費と、細切れになった経路情報によるインターネット全体のルーティングテーブルの肥大化を招いていました。

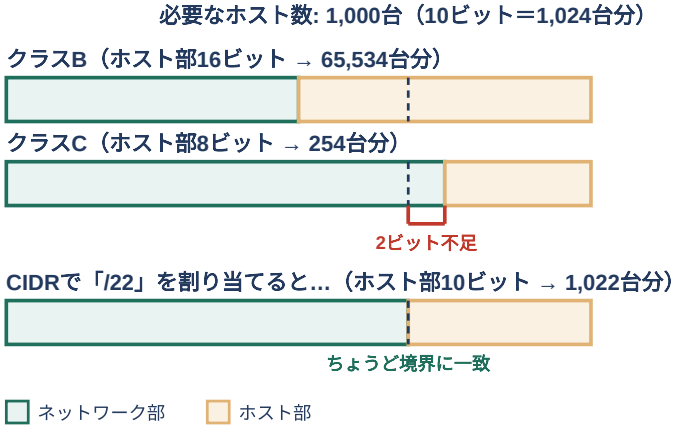
4.3　CIDR表記

サブネットマスクを「/24」のように、ネットワーク部のビット数だけで簡潔に表す方法が、CIDR表記です。

4.2節で見たクラスフルアドレッシングの「3種類の固定長からしか選べない」という制約を取り払い、ネットワーク部の長さを1ビット単位で自由に決められるようにしたのが、CIDR（Classless Inter-Domain Routing）という表記法です※3。先ほどの1,000台の組織の例でいえば、クラスの枠に縛られず「/22」（ホスト部が10ビットなので、2の10乗から2を引いた1,022台分）というちょうどよい大きさを割り当て

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

られるようになり、無駄なアドレス消費が大幅に減りました。

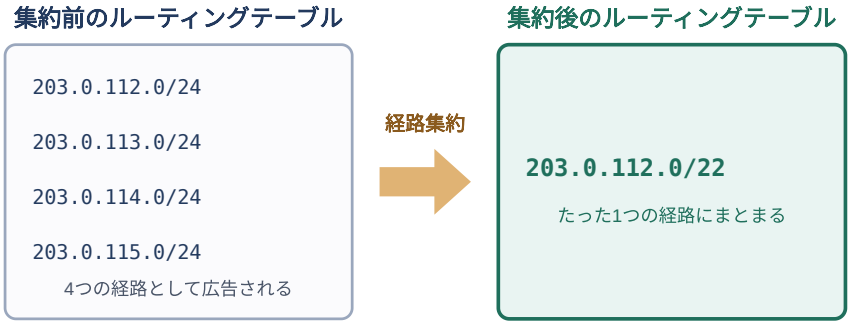


ビット幅で揃えて比較すると、クラスフルでは大きすぎるか小さすぎるかの二択だったが、CIDRならちょうどよい大きさを割り当てられるとわかる

表記そのものはシンプルで、サブネットマスクのうち、先頭から1が何ビット連続しているかを、/に続けて書くだけです。たとえば`255.255.255.0`は、4.2節で見たとおり2進数で`11111111.11111111.11111111.00000000`、つまり先頭から24ビットが1なので、`/24`と表記します。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

この表記のおかげで、組織の規模に合わせて必要な分だけのアドレス空間を無駄なく割り当てられるようになっただけでなく、連続する複数のネットワークをまとめて1つの経路として広告できるようにもなりました（経路集約）。たとえば`203.0.112.0/24`から`203.0.115.0/24`までの4つの連続したネットワークは、`203.0.112.0/22`という1つの経路にまとめて広告できます。



経路集約: 連続する複数のネットワークを、1つの経路にまとめて広告できる

この経路集約により、インターネット全体のルーティングテーブルの肥大化も抑えられるようになりました。4つの経

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

路を別々に覚えておく代わりに、1つの経路だけを覚えておけばよいからです。

4.4　デフォルトゲートウェイ

宛先がどこにあるか自分では判断できないとき、「とりあえずここへ送っておけばよい」という、困ったときのための宛先として設定しておく機器のアドレスが、デフォルトゲートウェイです。

インターネットのどこかにある、見てみたいWebサイトにアクセスしたい、ということがあるでしょう。けれど、インターネット上に存在する無数の宛先それぞれについて、「そこへはどの経路で届ければよいか」を機器自身があらかじめ全部知っておくことなど、現在のネットワークの数を考えれば到底できません。

では、どうすればよいのでしょうか。ある機器が通信をしようとするとき、宛先のIPアドレスが自分と同じネットワーク部を持っていれば、直接やり取りできます。しかし、宛先が別のネットワークに属している場合、その機器は「ここか

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

ら先、どうやって届ければよいか」を自分では判断できません。

そこで、「宛先がわからないときは、とりあえずここへ送っておけばよい」という、困ったときのための宛先をあらかじめ1つ決めておく、という発想が生まれました。あらかじめ設定しておいた、自分のネットワークの出入り口となる機器（多くの場合ルータ）に、その通信を丸ごと預けてしまえばよいのです。この「困ったときの宛先」の役割を果たす機器のIPアドレスを、デフォルトゲートウェイと呼びます。

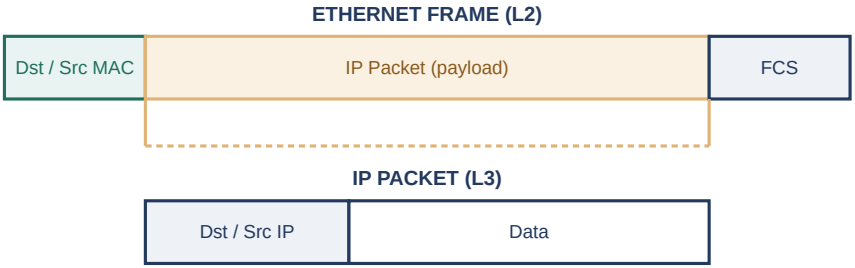
序章で見た「よそのネットワークともつながりたい」という欲求と、それに応えるルータの役割は、まさにこのデフォルトゲートウェイという形で、日々の通信の中に現れています。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

4.5　MAC+IPの統合：エンドツーエンド通信はどう成立するか

実際の通信は、IPアドレスで「最終的な行き先」を、MACアドレスで「今まさに渡す相手」を指定する、この2つのアドレスを組み合わせることで成立しています。

第3章で見たイーサネットフレームと、この章で見たIPパケットは、別々の層のものですが、実際にはIPパケットがまるごとイーサネットフレームの中身（ペイロード）として運ばれる、という入れ子の関係になっています。



L3のIPパケットが、L2のイーサネットフレームの中身として運ばれる

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

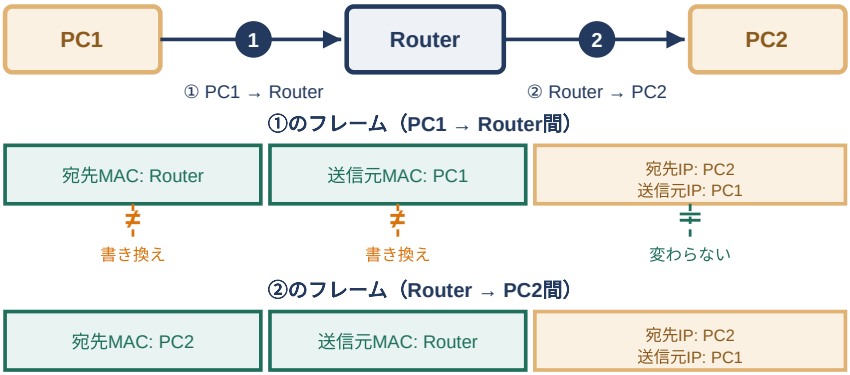
ここで疑問が浮かびます。送信したい相手のIPアドレスはわかっていても、フレームの宛先には本来MACアドレスを書く必要があります。IPアドレスから、対応するMACアドレスをどうやって知ればよいのでしょうか。

この「IPアドレスから、対応するMACアドレスを問い合わせる」役割を果たすのが、ARP（Address Resolution Protocol）という仕組みです。1982年にRFC 826として標準化されており※4、同じネットワークの中に向けて「このIPアドレスを持っているのは誰ですか」と問いかけ、該当する機器が自分のMACアドレスを返答する、という手順になっています。宛先が別のネットワークにある場合は、4.4節で見たデフォルトゲートウェイのIPアドレスに対してこのARPを行い、得られたMACアドレスを最初のフレームの宛先MACアドレスとして使います。ARPの具体的な手順は第7章で改めて扱います。

ここで押さえておきたいのは、通信の最初から最後まで、宛先のIPアドレスはずっと変わらない一方、フレームの宛先MACアドレスは、ルータを1つ越えるたびに「次にどの機器へ渡すか」に応じて書き換えられる、という点です。IPアドレスが旅全体の最終目的地を、MACアドレスがそのときど

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

きの「次の1歩」を指し示している、と考えると、両者の役割の違いがつかみやすくなります。



IPアドレスは変わらず、MACアドレスだけがルータを越えるたびに書き換えられる（第5章以降で学ぶ複数ホップの中継は、ここでは扱わない）

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

> MACアドレスとIPアドレスの役割分担から少し話を戻しますが、クラウドの仮想ネットワーク（AzureのVNet、AWSのVPCなど）を作成するときも、必ずCIDR表記でアドレス範囲を指定します。たとえば「10.0.0.0/16のVNetの中に、10.0.1.0/24と10.0.2.0/24という2つのサブネットを作る」といった設計は、この章で見てきたサブネッティングとCIDR表記の考え方をそのまま延長したものです。オンプレミスのネットワーク設計とクラウドの仮想ネットワーク設計は、見た目の操作画面こそ違いますが、根底にある考え方は共通しています。

参考文献

- ※1: RFC 1918, *Address Allocation for Private Internets* — https://www.rfc-editor.org/rfc/rfc1918.html
- ※2: RFC 791, *Internet Protocol* — https://www.rfc-editor.org/rfc/rfc791.html

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

- ※3: RFC 4632, *Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan* — https://www.rfc-editor.org/rfc/rfc4632.html
- ※4: RFC 826, *An Ethernet Address Resolution Protocol* — https://www.rfc-editor.org/rfc/rfc826.html

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

第5章　ルーティング

幕間　経路が増えて面倒になった話

前の章までで見てきたルータは、まだ1〜2台程度の、こぢんまりとしたネットワーク同士をつなぐ役回りでした。ところが、つながる相手が増えてくると、様子が変わってきます。

拠点が増え、ルータが3台、4台と増えていくと、「どのネットワーク宛てのパケットは、どのルータに投げればよいか」という経路の情報を、管理者が1つずつ手作業で設定し続けなければならなくなります。ルータが1台増えるたびに、既存のルータすべての設定を見直す必要が出てくることもあり、経路が増えるほど、この作業はどんどん面倒になっていきます。

さらに話は社内にとどまりません。組織同士がインターネットでつながり合うようになると、自分の組織の中だけで完結しないネットワーク同士の付き合い方も必要になってきます。大学同士、企業同士が回線を持ち寄り、IX（Internet

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

Exchange、インターネットエクスチェンジ）と呼ばれる相互接続点に集まって経路をやり取りする、という光景も生まれました。

こうして、ネットワークは大学や企業といった組織の垣根を越えて、世界規模でつながる1つの大きな集合体へと育っていきます。この章では、「経路を自動で管理する仕組み」と、「組織をまたいで経路をやり取りする仕組み」の両方を見ていきます。

5.1　ルーティングテーブルの仕組み

ルータは、パケットの宛先IPアドレスを見て、「次にどのルータ（または機器）へ渡せばよいか」をルーティングテーブルという表で判断しています。

第4章で見たように、宛先が別のネットワークにある通信は、デフォルトゲートウェイであるルータに預けられます。では、そのルータは預かった荷物をどこへ送ればよいのでしょうか。ルータは内部に、ルーティングテーブル（経路表）と呼ばれる一覧を持っており、そこに書かれた情報をもとに転送先を決めています。

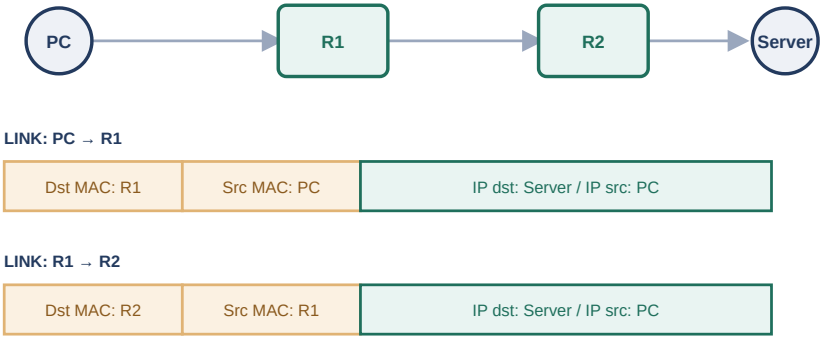
© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

ルーティングテーブルの主な項目

- 宛先ネットワーク: `192.168.10.0/24` のような、CIDR表記のネットワークアドレス
- ネクストホップ: そのネットワーク宛ての通信を、次にどの機器（多くは隣接するルータ）に渡すか
- 出力インターフェース: どの物理的な（または論理的な）出口から送り出すか

ある宛先IPアドレスが、ルーティングテーブル上の複数のエントリに部分的に一致することもあります。たとえば `192.168.10.5` という宛先は、`192.168.0.0/16` にも `192.168.10.0/24` にも当てはまります。このような場合、ルータはより長いプレフィックス（より狭い範囲を指す、より詳細なエントリ）を優先します。これをロンゲストプレフィックスマッチ（最長一致）と呼びます。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode



ホップを重ねてもIPアドレスは変わらず、MACアドレスだけが1区間ごとに書き換わる

第4章の終わりで触れた「IPアドレスが旅全体の最終目的地を、MACアドレスがそのときどきの次の1歩を指し示す」という関係は、ルータが何台連なっても変わりません。ルータは受け取ったパケットのIPアドレスはそのままに、ルーティングテーブルを引いてネクストホップを決め、そのネクストホップ宛ての新しいフレーム（宛先MACアドレスを次のホップのものに書き換えたフレーム）に包み直して送り出しているのです。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

5.2　スタティックルーティングとデフォルトルート

管理者があらかじめ経路を1つずつ手で設定しておく方式が、スタティックルーティング（静的経路制御）です。

ルーティングテーブルへの経路の登録方法には、大きく分けて2種類あります。1つは、管理者が「このネットワーク宛てなら、このネクストホップへ」という情報を、コマンドなどで1つずつ手作業で登録する方法です。これをスタティックルーティングと呼びます。拠点数が少なく、ネットワーク構成があまり変化しない環境では、シンプルで見通しがよい方法です。

スタティックルーティングでよく使われる仕組みに、デフォルトルートがあります。ルーティングテーブルのどのエントリにも一致しない宛先が現れたとき、代わりに使われる「その他すべて」用のエントリで、`0.0.0.0/0`という、あらゆるIPアドレスに一致する特別なネットワーク表記で登録します。多くの場合、インターネット向けの出口となるルータ

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

へのネクストホップとして設定され、「知らない宛先は、とりあえずここに投げておけばよい」という役割を果たします。

5.3　ダイナミックルーティングという考え方

> **経路情報をルータ同士が自動でやり取りし合い、ルーティングテーブルを自動的に構築・更新する仕組みが、ダイナミックルーティング（動的経路制御）です。**

冒頭の幕間で見たとおり、ルータの台数が増えるほど、スタティックルーティングの手作業には限界が見えてきます。設定漏れや入力ミスが起きやすくなるだけでなく、どこかの回線が切れて経路が使えなくなったときも、管理者が気づいて手動で経路を書き換えるまで、通信は復旧しません。

そこで登場するのが、ルータ同士がお互いの持つ経路情報を自動的に交換し合い、ネットワーク全体の変化に応じてルーティングテーブルを自ら更新していく、ダイナミックルーティングという仕組みです。ダイナミックルーティングを実

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

現する具体的な手順（プロトコル）にはいくつかの方式があり、大きくは次の2つの考え方に分けられます。

ダイナミックルーティングの2つの考え方

- ディスタンスベクタ型: 隣接するルータに「このネットワークまでの距離（ホップ数など）」を教え合い、伝言ゲームのように経路情報を伝播させる
- リンクステート型: すべてのルータが、ネットワーク全体の接続構成（トポロジー）そのものを把握し、各ルータが自分自身で最短経路を計算する

次の5.4節で扱うOSPFは、後者のリンクステート型の代表例です。

5.4　OSPFの基本的な特徴（設定・エリア設計は扱わない）

OSPF（Open Shortest Path First）は、組織内のネットワーク（1つの管理単位）で広く使われている、リンクステート型のダイナミックルーティングプロトコルです。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

OSPFの現行仕様は、1998年にRFC 2328として標準化されています※1。隣接するルータ同士がHelloパケットと呼ばれるメッセージを定期的にやり取りして互いの存在を確認し合い、その後、ネットワーク全体の接続構成の情報（リンクステート情報）を、すべてのルータに行き渡らせます。

OSPFの特徴

- リンクステート型: 各ルータが、ネットワーク全体の接続構成を把握したうえで、自分自身で最短経路を計算する（計算にはダイクストラ法というアルゴリズムを使う）
- コストというメトリック: 回線の帯域幅などをもとにした「コスト」の合計が最も小さい経路を、最短経路として選ぶ
- 収束が速い: 経路に変化があった場合、その変化の情報だけをすぐに周囲へ伝えるため、経路表が新しい状態に落ち着くまでの時間が短い
- IGP（Interior Gateway Protocol）: 1つの組織・1つの管理者が管理する範囲（後述するAS）の「内部」で使うことを想定したプロトコル

OSPFには、大規模なネットワークを複数の「エリア」に分割し、経路情報の伝播範囲を制限することで負荷を抑える

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

仕組みも用意されています（すべてのエリアは、バックボーンと呼ばれる「エリア0」を中心に接続される、という設計です）。ただし、エリア設計や実際の設定手順は本書の範囲を超えるため、ここでは「そういう仕組みがある」という程度に留めておきます。

5.5　組織間接続からBGPへ（iBGP/eBGP、比較優位性）

異なる組織同士のネットワーク（AS）をつなぐには、OSPFのような組織内向けのプロトコルではなく、組織間の経路交換に特化したBGPというプロトコルが使われます。

OSPFは、1つの組織が管理する範囲の内部で使うプロトコルでした。では、大学と大学、企業とインターネットサービスプロバイダ（ISP）のように、それぞれ独立して管理されている組織同士のネットワークをつなぐときは、どうすればよいのでしょうか。

ここで登場するのが、AS（Autonomous System、自律システム）という単位です。ASとは、単一の技術的な管理下に

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

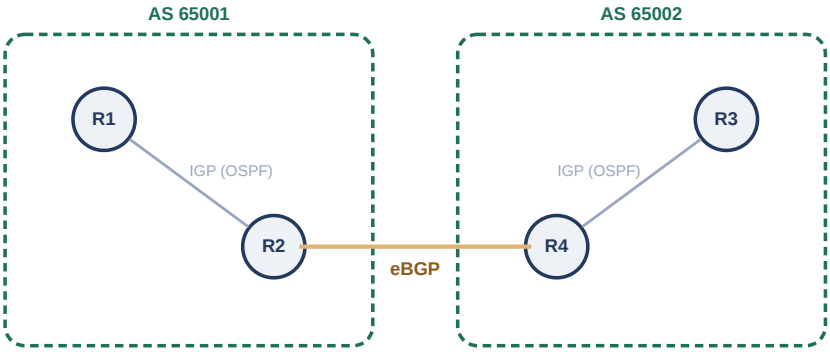
置かれ、内部では共通のルーティング方針（OSPFのようなIGP）で経路を制御しているネットワークのまとまりを指し、それぞれのASには、世界で重複しないAS番号が割り当てられます※2（本書の例で使う`65001`のような番号は、実運用でも試験・社内利用向けに予約されているプライベートAS番号の範囲です）。組織同士のネットワークをつなぐということは、つまり異なるASをつなぐということです。

異なるASの間では、それぞれの組織が独自の運用方針（どの相手を優先してよいか、どの経路は使いたくないか、など）を持っているため、OSPFのような単純な「コストが一番小さい経路を選ぶ」だけでは不十分です。そこで、AS間の経路交換専用に設計されたBGP（Border Gateway Protocol）というプロトコルが使われます※3。BGP自体は1989年に最初のバージョンが標準化されており、現在広く使われているBGP-4は1994年に初めて標準化された後、2006年にRFC 4271として改訂されています。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

iBGPとeBGP

- eBGP（external BGP）：異なるAS同士の境界ルータの間で経路をやり取りする
- iBGP（internal BGP）：同じASの中で、BGPの経路情報をルータ間で伝達するために使う



AS内部はIGP、AS間はeBGPという役割分担

OSPFが「コストという単一の物差しで、最短経路を機械的に選ぶ」ことを得意とするのに対し、BGPは「経路そのものの情報（経由するASの並びなど）をもとに、組織ごとの事情に応じて経路を選び分けられる」という、方針（ポリシー）に基づいた柔軟な経路選択を得意とします。この違いこそが、OSPFのようなIGPと、BGPのようなEGP（Exterior

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

Gateway Protocol）を使い分ける理由です。冒頭で触れたIXは、まさにこの多数のASが集まってBGPで経路をやり取りする、物理的・論理的な集合場所として機能しています。

> クラウドが提供するルートテーブル（Azureのルートテーブル、AWSのルートテーブルなど）も、宛先ネットワークとネクストホップの組み合わせで経路を管理するという、この章で見た考え方をそのまま踏襲しています。また、オンプレミス環境とクラウドを専用線でつなぐAzure ExpressRouteやAWS Direct Connectでは、双方の境界ルータの間でeBGPセッションを確立し、経路情報を交換する構成が広く使われています※4※5。オンプレミスとクラウドという異なる管理単位をBGPでつなぐという意味で、この章で見たAS間接続と同じ発想の延長線上にあります。

参考文献

- ※1: RFC 2328, *OSPF Version 2* — https://www.rfc-editor.org/rfc/rfc2328.html

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

- ※2: RFC 1930, *Guidelines for creation, selection, and registration of an Autonomous System (AS)* — https://www.rfc-editor.org/rfc/rfc1930.html
- ※3: RFC 4271, *A Border Gateway Protocol 4 (BGP-4)* — https://www.rfc-editor.org/rfc/rfc4271.html
- ※4: Microsoft Learn, *ExpressRoute: Routing requirements* — https://learn.microsoft.com/en-us/azure/expressroute/expressroute-routing
- ※5: AWS Direct Connect User Guide, *Routing policies and BGP communities* — https://docs.aws.amazon.com/directconnect/latest/UserGuide/routing-and-bgp.html

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

第6章　VLAN

6.1　VLANが生まれた背景（コスト圧力と論理分割）

VLAN（Virtual LAN）は、1台の物理スイッチを、まるで複数台の別々のスイッチであるかのように論理的に分割する技術です。

序章では、機器が増えるにつれてコスト圧力が高まり、スイッチを論理的に分割したくなる場面が出てくる、という話をしました。第3章で見たように、L2スイッチはコリジョンドメインをポートごとに分割してくれますが、ブロードキャストドメインまでは分割してくれません。部署やチームが違う機器同士を、ブロードキャストドメインのレベルで分けたい場合、従来の発想では「部署ごとに専用のスイッチを1台ずつ用意する」しかありませんでした。

しかし、これは台数が増えるほどコストがかさみます。各部署の利用率が低くても、スイッチ自体は部署の数だけ必要

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

になりますし、部署間で人員の異動があるたびに、配線を物理的に付け替える手間もかかります。

VLANが解決した課題

- 部署・チームごとにスイッチを1台ずつ用意すると、台数が増えるほどコストがかさむ
- 利用率の低い部署のためだけに、専用のスイッチ1台を遊ばせておくのはもったいない
- 人員の異動のたびに、物理的な配線を付け替えるのは手間がかかる

そこで、「1台の物理スイッチを、ソフトウェア的にいくつかの仮想的なスイッチとして扱えるようにしよう」という発想が生まれました。これがVLANです。1台のスイッチのポートを、VLAN 10、VLAN 20のようにグループ分けしておくことで、まるで別々のスイッチを部署ごとに用意したかのような効果を、1台のハードウェアだけで実現できます。

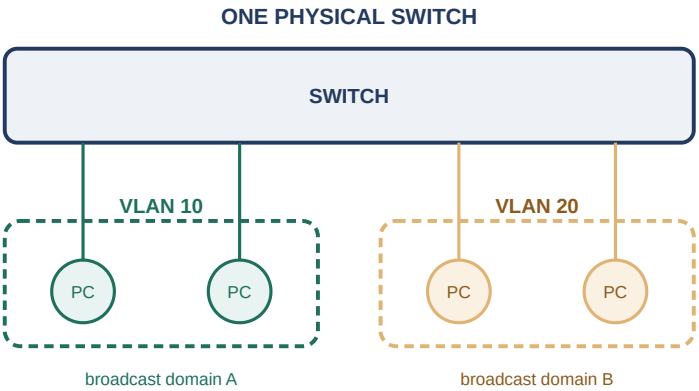
© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

6.2　実は…ルータと同じ「ブロードキャストドメインの分割」をL2で実現する技術

VLANの正体は、1台の物理スイッチの中に、複数の独立したブロードキャストドメインを作り出す仕組みです。

第3章では、「ブロードキャストドメインを分割できるのは、L3（ネットワーク層）で動作するルータだけです」という説明をしました。実は、この説明には続きがあります。VLANを使うことで、L2スイッチ1台だけでも、ブロードキャストドメインを複数に分割できてしまうのです。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode



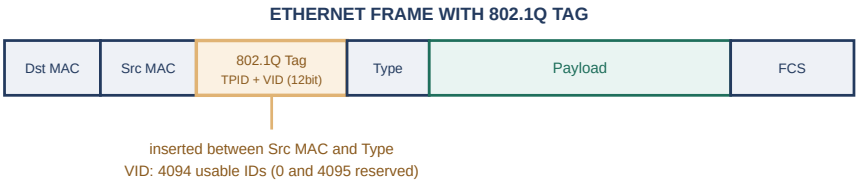
1台の物理スイッチが、2つの独立したブロードキャストドメインに分かれる

VLANが設定されたポートに届いたブロードキャストフレームは、同じVLANに属する他のポートにしか転送されません。異なるVLANに属するポート同士は、同じ物理スイッチにつながっていても、まるで別のスイッチにつながっているかのように、ブロードキャストが届かない別々の範囲として扱われます。

この仕組みを実現しているのが、IEEE 802.1Qという標準規格です※1。複数のスイッチにまたがってVLANを構成し、1本のリンクで複数のVLANのフレームをやり取りする際には（詳しくは次節で見ます）、イーサネットフレームの送信元MACアドレスとタイプフィールドの間に、4バイトのVLAN

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

タグを挿入し、そのフレームがどのVLANに属するかを保持します。



802.1Qタグは、送信元MACアドレスとタイプの間に挿入される

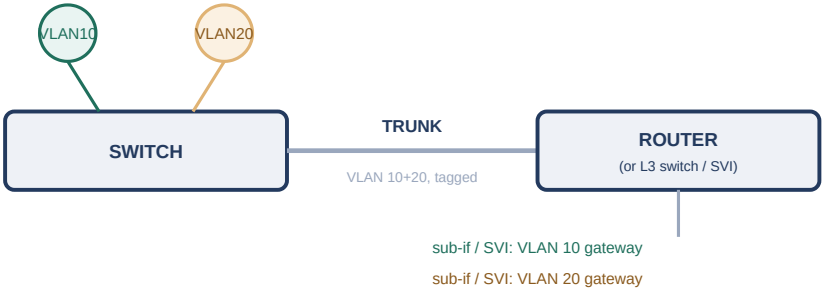
このタグの中には、12ビットのVLAN識別子（VID）が含まれており、理論上4096通りの値を表現できますが、0と4095は予約されているため、実際に使えるVLANの数は4094個です※1。スイッチは、このタグを見て「このフレームはどのVLANに属するか」を判断し、同じVLANのポートにだけ転送します。

6.3　VLAN間ルーティングとトランク

異なるVLAN同士は別々のブロードキャストドメインなので、VLANをまたいで通信するには、ルータ（またはL3スイッチ）による経路選択が必要です。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

複数のスイッチにまたがってVLANを構成する場合、スイッチ同士を結ぶ1本のリンクの中に、複数のVLANのフレームを混在させて送る必要が出てきます。この、複数VLANのタグ付きフレームを1本の物理リンクで同時に運ぶ接続を、トランクと呼びます。これに対して、1つのVLANにしか属さない、通常の機器を接続するためのポートは、アクセスポートと呼ばれます。



トランクで複数VLANを1本のリンクに乗せ、ルータ（またはL3スイッチ）が各VLANのゲートウェイになる

ここで、第5章で見た内容が生きてきます。異なるVLANは、それぞれ独立したブロードキャストドメイン、つまり別々のネットワーク（別々のサブネット）として設計される

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

のが普通です。ということは、VLAN 10にいる機器がVLAN 20にいる機器と通信したいときは、第5章で見たルーティングの出番だということです。具体的には、次の2つの構成のどちらかがよく使われます。

VLAN間ルーティングの構成パターン

- ルータ・オン・ア・スティック: ルータの1本の物理インターフェースに複数のVLANのトランクを接続し、ルータ側でVLANごとのサブインターフェースを作って、それぞれをそのVLANのデフォルトゲートウェイにする
- L3スイッチ（SVI）: スイッチ自体にルーティング機能を持たせ、VLANごとの仮想インターフェース（SVI、Switched Virtual Interface）をデフォルトゲートウェイとして動作させる

いずれの構成でも、「異なるVLAN（＝異なるブロードキャストドメイン）をまたぐ通信には、L3の機器によるルーティングが必要」という原則は変わりません。第3章で述べた「ブロードキャストドメインを分割できるのはルータだけ」という説明は、正確には「L2の世界の中でも、VLANを使えばブロードキャストドメインを分割できる。ただし、分割し

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

たVLAN同士をつなぎ直すには、結局ルータ（相当の機能）が必要になる」という形で、より正確な理解に置き換わったことになります。

> クラウドの仮想ネットワークにおける論理分離も、この章で見た発想の延長線上にあります。Azureのネットワークセキュリティグループ（NSG）やAWSのセキュリティグループは、VLANのようにL2のブロードキャストドメインそのものを分けるわけではありませんが、「同じ物理的な基盤の上に、論理的に独立した通信の境界を作る」という目的は共通しています。また、クラウドの仮想ネットワークをサブネット単位で分割する設計は、第4章で見たサブネッティングの考え方がそのまま活きている部分です。オンプレミスのVLAN設計とクラウドの論理分離は、実現手段こそ異なりますが、「物理的な統合と論理的な分離を両立させる」という狙いは同じです。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

参考文献

- ※1: IEEE 802.1Q-2022, *IEEE Standard for Local and Metropolitan Area Networks—Bridges and Bridged Networks* — https://standards.ieee.org/ieee/802.1Q/10323/

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

第7章　トランスポート層

7.1　サービスごとの「窓口」を区別する必要性 ― ポートという発想

IPアドレスが「どの機器（ホスト）宛てか」を示すのに対し、ポート番号は「その機器の中の、どのサービス宛てか」を区別するための番号です。

序章では、ファイル保管・印刷取りまとめなど、複数のサービスが貴重なサーバー1台にまとめられていく様子を見てきました。ここで新しい問題が生まれます。1台のサーバーが、Webサービスとメールサービスを同時に提供しているとき、届いた通信をどちらのサービス向けか、どうやって区別すればよいのでしょうか。

IPアドレスだけでは、この区別はできません。IPアドレスが指し示せるのは「どの機器か」までで、「その機器の中のどのプログラム（サービス）宛てか」までは指し示せないからです。そこで生まれたのが、ポート番号という「窓口」の

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

発想です。同じ機器の中に、サービスの数だけ窓口を用意しておき、通信の宛先としてIPアドレスとポート番号をセットで指定することで、「どの機器の、どのサービス宛てか」を一意に表せるようになります。

IPアドレスとポート番号の役割分担

- IPアドレス: 通信の相手が「どの機器か」を指し示す
- ポート番号: その機器の中で、通信が「どのサービス（プログラム）宛てか」を指し示す
- この2つを組み合わせることで、初めて「特定の機器の、特定のサービス」と通信できる

このポート番号を管理し、実際に窓口を開いたり閉じたりする役割を担うのが、この章のテーマであるトランスポート層（L4）です。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

7.2　TCPとUDPの違い

TCPは、届いたかどうかを保証し、順番も正しく並べ直してくれる信頼性重視のプロトコルで、UDPは、その保証を持たない代わりに軽量・高速なプロトコルです。

トランスポート層の代表的なプロトコルには、TCP（Transmission Control Protocol）とUDP（User Datagram Protocol）の2種類があります。TCPは、送ったデータが確実に届いたことを相手からの確認応答で確かめ、届いていなければ再送し、複数に分割されたデータも送信順に並べ直して渡す、という信頼性重視の設計になっています※1。一方のUDPは、こうした確認や並べ直しの仕組みを持たず、送りっぱなしにする代わりに、余計な手続きがない分、処理が軽くて速いという特徴があります※2。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

TCPとUDPの使い分け

- TCP: Webページの閲覧、メールの送受信、ファイル転送など、データが1バイトも欠けたり順番が入れ替わったりしては困る用途
- UDP: 動画・音声のストリーミング、オンラインゲーム、DNSの問い合わせなど、多少の欠落より速さや低遅延を優先したい用途

「信頼性が高いTCPだけを使えばよいのでは」と思うかもしれませんが、確認や再送のやり取りにはそのぶん時間がかかります。リアルタイム性が求められる通話や配信では、多少データが欠けても、待たされるよりはそのまま再生を続けたほうが体験がよいことも多く、あえてUDPが選ばれます。どちらが優れているというより、通信の性質に応じた使い分けです。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

7.3　ポート番号

ポート番号は16ビットの数値で、0〜1023番の「よく知られたポート」、1024〜49151番の「登録済みポート」、49152〜65535番の「動的・一時的なポート」の3つの範囲に分かれています。

ポート番号は16ビットの数値で、0番から65535番までの範囲を取ります。IANA（Internet Assigned Numbers Authority）が、この範囲を3つの区分に分けて管理しています※3。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

ポート番号の3つの区分

- 0〜1023番（よく知られたポート、Well Known Ports）：HTTPの80番、HTTPSの443番、SSHの22番、DNSの53番など、広く使われる主要サービスに割り当てられる
- 1024〜49151番（登録済みポート、Registered Ports）：特定のアプリケーション・サービス向けにIANAへ登録申請できる範囲
- 49152〜65535番（動的・一時的なポート、Dynamic/Ephemeral Ports）：クライアント側が通信のたびに一時的に使う、割り当て不要の範囲

普段、Webブラウザでサイトを閲覧するとき、サーバー側は80番や443番といった決まったポートで待ち受けていますが、クライアント側（自分のPC）は、通信のたびに動的・一時的なポートの範囲からランダムに1つを選んで使っています。この「サーバー側は決まった窓口番号で待ち、クライアント側はその都度空いている窓口を使う」という非対称な関係も、ポート番号の実際の使われ方を理解するうえで押さえておきたいポイントです。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

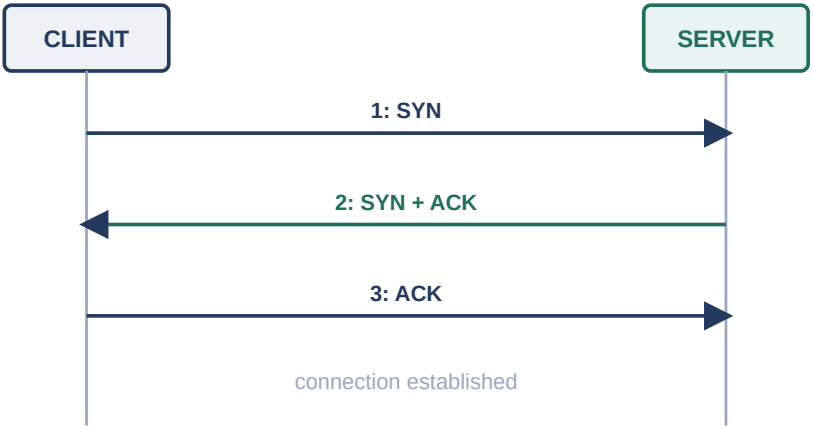
クラウドのネットワークセキュリティグループ（NSG）やセキュリティグループでフィルタ設定をするとき、TCP/UDPとポート番号（80番、443番など）を指定するのは、まさにこのポート番号の仕組みをそのまま使っています。また、クラウドのNATゲートウェイ（NAT自体の仕組みは第8章で扱います）では、多数の送信元機器が動的・一時的なポートを奪い合う結果、「ポートが枯渇してそれ以上新しい通信を開始できない」という実務上の問題が起こることがあり、ポート番号が有限の資源であることを意識させられる場面の1つです。

7.4　TCPの通信手順（3ウェイハンドシェイク）

TCPは通信を始める前に、SYN・SYN+ACK・ACKという3回のやり取りで、お互いに「これから通信を始めます」という合意を取り交わします。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

TCPが信頼性を保証できる理由の1つが、本格的なデータのやり取りを始める前に、必ず接続を確立する手続きを踏む点にあります。この手続きは、3回のやり取りから成るため、3ウェイハンドシェイクと呼ばれています※1。



3ウェイハンドシェイク: SYN → SYN+ACK → ACK

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

3ウェイハンドシェイクの流れ

- 1回目（SYN）：クライアントがサーバーに「接続を始めたい」という信号を送る
- 2回目（SYN+ACK）：サーバーが「了解した、こちらも接続を始めたい」と、受諾と自分からの要求を同時に返す
- 3回目（ACK）：クライアントが「了解した」と応答し、ここで両者の接続が確立する

この手続きによって、お互いが「相手が確かに存在し、通信を受け付ける準備ができている」ことを確認してから、本題のデータをやり取りできます。通信が終わるときにも、今度は双方向でFIN（終了要求）とACKを交わし合い、双方が合意のうえで接続を終了させる手順が定められています。

7.5　ARPとICMP

ARPとICMPは、厳密にはトランスポート層のプロトコルではありませんが、通信の診断・補助という役割から、この章でまとめて紹介します。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

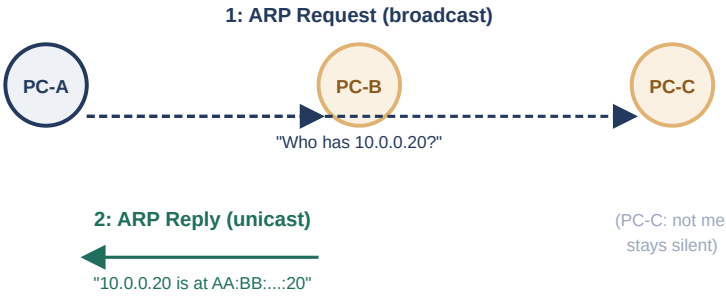
ここまでトランスポート層の話をしてきましたが、ARPとICMPは、第1章で見たOSI参照モデルのどの層にもすっきり収まる存在ではありません。ARPはIPアドレスとMACアドレスを橋渡しする、L2とL3の間の実務的な仕組みであり、ICMPはIPそのものの一部として動くL3寄りの仕組みです。どちらも「通信そのものを届ける」役割ではなく、「通信を成立させるための下準備をする」「通信の状態を診断する」という、縁の下の役割を担っているという共通点から、本書ではこの章にまとめて置いています。

ARP・ICMPの位置づけ

- ARP（Address Resolution Protocol）: IPアドレスから対応するMACアドレスを調べる、L2/L3の橋渡し役
- ICMP（Internet Control Message Protocol）: 通信の成否やエラーを報告する、IPに付随する診断・補助の仕組み

第4章では、ARPを「IPアドレスから、対応するMACアドレスを問い合わせる仕組み」として概念的に紹介しました。ここで、その具体的な手順を見ていきます。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode



ARP: ブロードキャストで問い合わせ、該当する機器だけがユニキャストで応答する

ARPは1982年にRFC 826として標準化されています※4。ある機器が、同じネットワーク内の別の機器のMACアドレスを知りたいとき、「このIPアドレスを持っているのはどなたですか」という問い合わせ（ARP Request）を、同じネットワーク内の全機器へブロードキャストで送ります。該当するIPアドレスを持つ機器だけが、「わたしです、MACアドレスはこれです」という応答（ARP Reply）を、問い合わせ元へユニキャスト（1対1）で返します。この問い合わせと応答の結果は、ARPテーブル（ARPキャッシュ）としてしばらくの間手元に保存されるため、同じ相手に何度も問い合わせる必要はありません。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

ICMPは、IPパケットが正しく届いたかどうかや、途中で何か問題が起きていないかを報告するための仕組みで、1981年にRFC 792として標準化されています※5。日常的によく使われる`ping`コマンドは、ICMPのエコー要求（Echo Request）を相手に送り、エコー応答（Echo Reply）が返ってくるかどうかで、相手との疎通を確認する仕組みです。また、`traceroute`（`tracert`）コマンドは、パケットの生存時間（TTL）を1ずつ増やしながら送り、途中の各ルータから返される「時間切れ」のICMPメッセージを利用して、宛先までの経路上のルータを1台ずつ明らかにする、という応用的な使い方をしています。

参考文献

- ※1: RFC 9293, *Transmission Control Protocol (TCP)* — https://www.rfc-editor.org/rfc/rfc9293.html
- ※2: RFC 768, *User Datagram Protocol* — https://www.rfc-editor.org/rfc/rfc768.html
- ※3: RFC 6335, *Internet Assigned Numbers Authority (IANA) Procedures for the Management*

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

of the Service Name and Transport Protocol Port Number Registry — https://www.rfc-editor.org/rfc/rfc6335.html

- ※4: RFC 826, *An Ethernet Address Resolution Protocol* — https://www.rfc-editor.org/rfc/rfc826.html
- ※5: RFC 792, *Internet Control Message Protocol* — https://www.rfc-editor.org/rfc/rfc792.html

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

第8章　アクセス制御とNAT

8.1　ACLという考え方

ACL（Access Control List、アクセス制御リスト）は、「どの通信を許可し、どの通信を拒否するか」をルールの並びとして定義し、ルータやファイアウォールに適用する仕組みです。

序章では、組織同士がネットワークでつながるようになると、「入れたくない相手を締め出す」という新しい悩みが生まれる様子を見てきました。つながること自体は便利ですが、あらゆる通信を無条件に受け入れてしまうと、悪意のある通信も素通りしてしまいます。そこで生まれたのが、通信を選別するという発想です。

ACLは、送信元IPアドレス・宛先IPアドレス・プロトコル（TCP/UDPなど）・ポート番号といった条件をもとに、「この条件に一致する通信は許可（permit）」「この条件に一致する通信は拒否（deny）」というルールを、上から順に並べたリストです。ルータやファイアウォールは、届いた通信

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

を上から順にルールと照合し、最初に一致したルールに従って通信を通すか止めるかを決めます。

ACLの基本的な考え方

- ルールは上から順に評価され、最初に一致したルールが適用される（それより下のルールは評価されない）
- 明示的にどのルールにも一致しなかった通信は、多くの実装で暗黙的に拒否される
- 送信元/宛先IPアドレス、プロトコル、ポート番号などを条件として組み合わせられる

ここで注意が必要なのは、伝統的なACLの多くは、通信の「行き」と「帰り」を自動的には結び付けてくれない、ステートレス（状態を持たない）な仕組みだという点です。たとえば、社内からWebサーバーへのアクセスを許可するルールを作っただけでは不十分で、その応答（帰りの通信）を許可するルールも別途用意しないと、通信が成立しません。これに対し、現代のファイアウォールの多くは、「行きの通信を許可すれば、対応する帰りの通信は自動的に許可する」というステートフル（状態を保持する）な処理を備えており、設定の手間と安全性の両方が改善されています。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

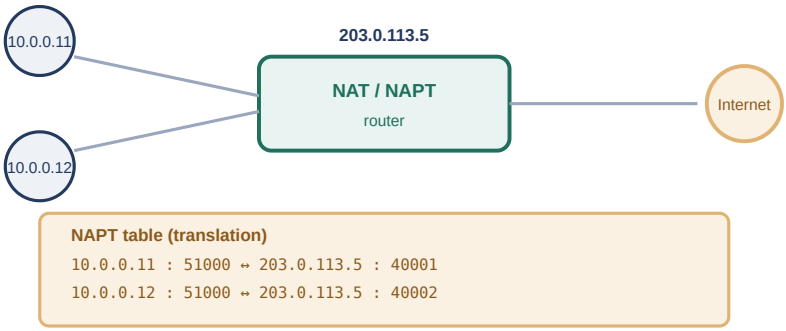
8.2　NAT/NAPTの仕組み

NATは、プライベートIPアドレスとグローバルIPアドレスを変換する仕組みで、NAPTは、その変換にポート番号も組み合わせることで、複数の機器が1つのグローバルIPアドレスを共有できるようにする仕組みです。

第4章では、`192.168.x.x`や`10.x.x.x`といったプライベートIPアドレスが、インターネットには直接ルーティングされないという話をしました。では、プライベートIPアドレスを持つ社内の機器は、どうやってインターネット上のサーバーと通信しているのでしょうか。ここで働いているのが、NAT（Network Address Translation、ネットワークアドレス変換）です※1。

NATには、大きく分けて2つの方式があります。1つは、プライベートIPアドレス1つに対して、グローバルIPアドレス1つを対応させるだけの、単純なNAT（Basic NAT）です。もう1つは、第7章で見たポート番号も組み合わせて変換する、NAPT（Network Address Port Translation）※1で、「IPマスカレード」とも呼ばれます。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode



NAPT: 複数のプライベートIPアドレスを、ポート番号込みで1つのグローバルIPアドレスに変換する（同じローカルポート51000同士も、グローバル側では別ポートに割り当てて衝突を解決する）

NAPTでは、社内の機器がインターネットへ通信するたびに、「どのプライベートIPアドレス・ポート番号の組み合わせが、グローバル側のどのポート番号に対応しているか」という変換テーブルを、NATルータが動的に作成・管理します。この仕組みのおかげで、限られた数（多くの場合1個）のグローバルIPアドレスだけで、社内の多数の機器が同時にインターネットへアクセスできるようになります。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

8.3　豆知識：アドレス枯渇とCGNAT（読み飛ばし可）

IPv4アドレスは有限であり、世界的な枯渇に対応するため、通信事業者が加入者宅とインターネットの間にもう1段階のNATを挟む、CGNAT（Carrier-Grade NAT）という仕組みが使われることがあります。

IPv4アドレスは32ビットの数値なので、表現できる組み合わせは約43億個しかありません。インターネットに接続する機器が世界的に増え続けた結果、この数では足りなくなる「IPv4アドレス枯渇」が現実の問題になりました。

NAPTによって、1つのグローバルIPアドレスを社内の多数の機器で共有できるようになりましたが、それでも「1契約に1つのグローバルIPアドレス」を割り当てるだけの数が、通信事業者側で足りなくなってきています。そこで登場したのが、CGNAT（Carrier-Grade NAT、キャリアグレードNAT）です。CGNATは、契約者の自宅ルータとインターネットの間に、通信事業者がもう1段階のNATを挟むことで、複数の契約者に、同じグローバルIPアドレスを（ポート

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

番号を分け合う形で）共有させる仕組みです。契約者宅のルータとCGNAT設備の間では、`100.64.0.0/10`という、CGNAT専用に予約された共有アドレス空間が使われます※2。

CGNATは、枯渇問題への現実的な対処法である一方、契約者側からは「NATの中に、さらにNATがある」状態になるため、外部から契約者の機器へ直接アクセスする（特定のポート宛ての通信を外部から通せるようにする、いわゆるポート開放をする）ことが難しくなる、という副作用もあります。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

> クラウドのNSG（ネットワークセキュリティグループ）やセキュリティグループは、まさにこの章で見たステートフルなACLそのものです。インバウンドルールを1つ許可すれば、対応するアウトバウンドの応答は自動的に通過が許可される、という動作は、伝統的なステートレスACLとの違いとして意識しておくとよいでしょう。また、クラウド上の仮想マシンにグローバルIPアドレスを直接持たせず、ロードバランサやNATゲートウェイ経由でインターネットに出す構成も、この章で見たNAT/NAPTの考え方の延長線上にあります。

参考文献

- ※1: RFC 3022, *Traditional IP Network Address Translator (Traditional NAT)* — https://www.rfc-editor.org/rfc/rfc3022.html
- ※2: RFC 6598, *IANA-Reserved IPv4 Prefix for Shared Address Space* — https://www.rfc-editor.org/rfc/rfc6598.html

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

第9章　冗長化

9.1　冗長化はなぜ必要か

冗長化とは、通信経路や機器を複数用意しておくことで、そのうち1つが故障しても、全体としては通信が止まらないようにする設計のことです。

ここまでの章では、スイッチ・ルータ・ケーブルといった機器や配線が、正しく動いていることを前提に話を進めてきました。しかし、どんな機器やケーブルも、いつかは故障します。1台のスイッチ、1本のケーブル、1台のルータだけに頼った構成では、その1点が壊れただけで、ネットワーク全体、あるいはその先につながる利用者全員が影響を受けてしまいます。このような、そこが壊れると全体が止まってしまう1点のことを、単一障害点（SPOF、Single Point of Failure）と呼びます。

第3章では、スイッチ同士を複数のケーブルでループ状につなぐと、STP（スパニングツリープロトコル）によって、普段は使わない予備の経路を用意しつつ、論理的にはループ

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

のない状態を保つ仕組みを見ました。これは、L2における冗長化の一例です。この章では、L3（デフォルトゲートウェイ）や、物理的なリンクそのものについても、同じように「複数用意しておいて、1つが壊れても大丈夫にする」という発想を広げていきます。

9.2　HSRP/VRRPの仕組みと目的（ハートビート、無停止交換の実務価値）

HSRPやVRRPは、複数台のルータで1つの仮想的なデフォルトゲートウェイを共有し、実際に動いているルータが故障しても、機器側の設定を変えずに通信を継続できるようにする仕組みです。

第4章・第5章で見たように、機器はデフォルトゲートウェイのIPアドレスを1つだけ設定として持ちます。もし、そのデフォルトゲートウェイとして動いているルータが1台だけだと、そのルータが故障した瞬間に、社内から外へ出る通信がすべて止まってしまいます。かといって、機器ごとに「ルータAが落ちたらルータBに手動で設定を変える」というのは現実的ではありません。

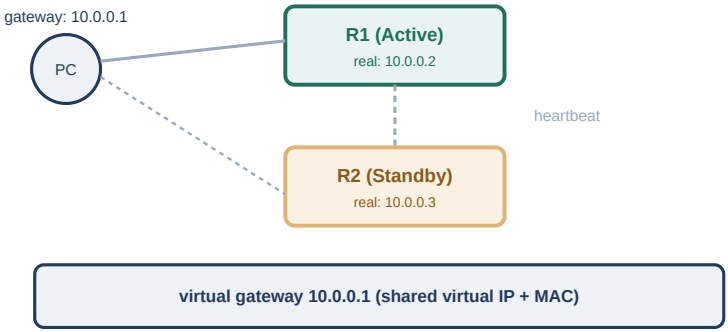
© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

そこで使われるのが、HSRP（Hot Standby Router Protocol）やVRRP（Virtual Router Redundancy Protocol）といった、FHRP（First Hop Redundancy Protocol、最初のホップの冗長化プロトコル）です。HSRPはCisco社が開発した方式でRFC 2281として文書化されており※1、VRRPはIETFが標準化した方式で、現行版はRFC 9568です※2。

HSRP/VRRPの仕組み

- 複数台のルータが、共有の仮想IPアドレス・仮想MACアドレスを持つ「仮想ルータ」を演じる
- 機器側のデフォルトゲートウェイには、この仮想IPアドレスだけを設定しておく
- 参加するルータの間では、ハートビート（生存確認のための短い間隔のメッセージ）が定期的にやり取りされ、転送を担当するルータが動いているかどうかが監視される
- 実際に転送を担当するルータ（アクティブ/マスター）が応答しなくなると、待機していたルータ（スタンバイ/バックアップ）が仮想IPアドレス・仮想MACアドレスを引き継ぎ、転送を肩代わりする

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode



2台のルータが仮想的な1つのデフォルトゲートウェイを演じる

この仕組みの実務上の価値は、故障時の自動切り替えだけにとどまりません。たとえば、ルータのソフトウェア更新やハードウェア交換をしたいとき、アクティブ側を意図的にスタンバイへ切り替えてから作業すれば、機器側には何の設定変更もさせずに、無停止（またはごく短時間の停止）でメンテナンスを行えます。これは、日々の運用の中で地味ながら大きな価値を持つ使い方です。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

9.3　LAGの仕組みと「同一論理スイッチ内限定」の罠

LAG（Link Aggregation Group）は、複数の物理リンクを束ねて1本の論理リンクとして扱う技術で、帯域の拡大と、リンク単位での冗長化の両方を実現します。

第3章で触れたように、機器同士を複数のケーブルでつなぐこと自体は、冗長化の基本形です。ただし、単純にケーブルを複数本つなぐだけでは、L2スイッチはループとみなしてSTPでブロッキングしてしまい、せっかくの複数本のうち1本しか実際には使われません。そこで、複数の物理リンクを、あたかも1本の太い論理リンクであるかのように束ねる技術が、LAGです。IEEE 802.1AXとして標準化されており※3、実際にリンクを束ねる制御にはLACP（Link Aggregation Control Protocol）というプロトコルが使われます。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

LAGが持つ2つの効果

- 帯域の拡大: 束ねた本数分、理論上の合計帯域が増える
- リンク単位の冗長化: 束ねた中の1本が切れても、残りのリンクで通信を継続できる（STPのブロッキングによる切り替えより高速）

ここで注意したいのが、伝統的なLAGは「1台の機器と、もう1台の機器」という、ちょうど2台のペアの間でしか組めない、という制約です。つまり、スイッチAから出た複数本のケーブルを束ねてLAGにする場合、その先はすべて「スイッチB」という同一の1台（に見える論理的な単位）につながっていなければなりません。スイッチAから出た複数本のケーブルを、物理的に別々の2台のスイッチ（スイッチBとスイッチC）に分けて、それでも1つのLAGとして扱う、ということは、伝統的なLAGの仕組みだけでは実現できないのです。

この制約を乗り越えるために、複数の物理スイッチを、LAGの相手から見て「1台の論理スイッチ」であるかのように振る舞わせる技術（スタッキング、MC-LAGやvPCなど、メーカーによって呼び方が異なる仕組み）が使われることが

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

あります。これにより、スイッチ本体そのものが1台丸ごと故障しても、LAGを組んだ相手から見れば「複数本のうちの何本かが切れただけ」として扱われ、通信を継続できるようになります。「LAGを組んでいるから安心」と思っていても、実はその先が物理的に1台のスイッチに集約されている、という構成では、そのスイッチ自体の故障がSPOFのまま残ってしまう、という点は見落としやすい罠です。

9.4　まとめ：2層・3層ネットワークアーキテクチャという設計の型

第3章から本章までで積み上げてきた要素は、アクセス層・ディストリビューション層・コア層という階層に機器を配置する、2層・3層ネットワークアーキテクチャという定番の設計の型に集約されます。

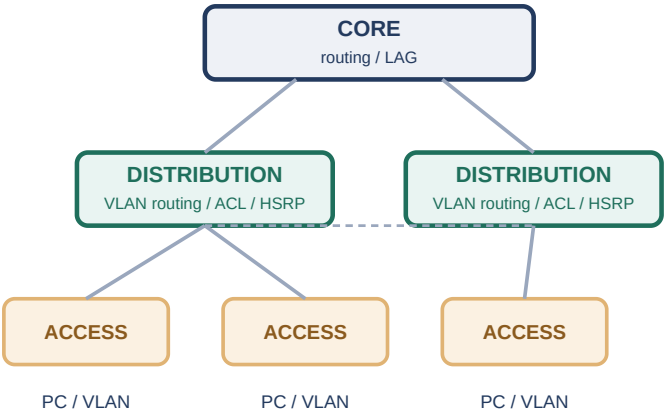
第3章のスイッチングから始まり、IPアドレス、ルーティング、VLAN、トランスポート層、ACL/NAT、そしてこの章の冗長化まで、ここまでの内容を振り返ってみましょう。実際の企業ネットワークでは、これらの要素は、大きく3つの階層に整理されるのが一般的です。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

3層ネットワークアーキテクチャの各層

- アクセス層: PCやサーバーなど末端の機器が実際に接続される層（第3章のスイッチ、第6章のVLAN）
- ディストリビューション層: 複数のアクセス層スイッチを集約し、VLAN間ルーティング（第6章）やACL（第8章）によるポリシー適用を行う層。多くの場合、この層の機器がHSRP/VRRP（本章）でデフォルトゲートウェイの冗長化を担う
- コア層: ディストリビューション層同士を高速に結ぶ、ネットワークの背骨にあたる層。第5章のルーティングプロトコルや、本章のLAGによる高速な経路切り替えが重要になる

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode



3層ネットワークアーキテクチャ: コア → ディストリビューション → アクセス

小規模なネットワークでは、ディストリビューション層とコア層を1つにまとめた2層構成が使われることも多く、規模や要件に応じて、この型は柔軟に使い分けられます。いずれにしても、「末端の接続」「ポリシーの適用」「高速な中継」という役割を階層として分離し、各階層・各機器を冗長化しておく、という設計の型そのものは共通しています。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

ここまでの章で見てきた要素を、この3層構造に当てはめて振り返ってみましょう。第3章のMACアドレステーブルとコリジョンドメインはアクセス層の基礎、第4章・第5章のIPアドレスとルーティングはディストリビューション層・コア層をまたぐ通信の基盤、第6章のVLANはアクセス層〜ディストリビューション層の論理分割、第7章のポート番号はエンドツーエンドの通信を成立させる最後のピース、第8章のACL/NATはディストリビューション層でのポリシー適用の実例、そして本章の冗長化は、これら全部の層に共通して求められる「壊れても止まらない」ための設計です。個々の技術要素がバラバラに存在するのではなく、この階層構造の中でどう組み合わさるかを意識すると、実際のネットワーク構成図の見え方が大きく変わってきます。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

> クラウドの世界では、物理的な機器の冗長化に相当する考え方が、アベイラビリティゾーン（AZ、独立した1つ以上のデータセンターからなる単位）への分散という形で現れます。1つのAZが丸ごと使えなくなっても、別のAZに同じ構成のリソースを用意しておけば、サービス全体は継続できます。また、ロードバランサは、この章で見たHSRP/VRRPと似た役割を果たします。利用者は常に1つの窓口（ロードバランサのアドレス）にアクセスするだけで、背後にある複数のサーバーのうちどれが実際に応答しているかを意識する必要がありません。オンプレミスの「仮想ゲートウェイを複数台のルータで演じる」という発想と、クラウドの「複数台のサーバーを1つの窓口の背後に隠す」という発想は、根底にある考え方が共通しています。

参考文献

- ※1: RFC 2281, *Cisco Hot Standby Router Protocol (HSRP)* — https://www.rfc-editor.org/rfc/rfc2281.html

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

- ※2: RFC 9568, *Virtual Router Redundancy Protocol (VRRP) Version 3 for IPv4 and IPv6* — https://www.rfc-editor.org/rfc/rfc9568.html
- ※3: IEEE 802.1AX-2020, *IEEE Standard for Local and Metropolitan Area Networks—Link Aggregation* — https://standards.ieee.org/ieee/802.1AX/6768/

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

第10章　DHCPとDNS

幕間　名前を教え合う話

小さな組織の中でネットワークを使い始めた頃は、機器の数もそれほど多くありません。「このIPアドレスはファイルサーバー」「あのIPアドレスは経理部の複合機」といった対応関係は、機器を管理している数人が頭の中で覚えているか、せいぜい1枚の一覧表を共有しておけば十分でした。実際、コンピュータ自身も、`hosts`ファイルと呼ばれる簡単な対応表を手元に持たせておくだけで、名前とIPアドレスの対応づけを済ませることができました。

しかし、組織の外にある無数のコンピュータともやり取りしたい、という段階になると、この方法は破綻します。世界中の誰かがサーバーを新設したり、IPアドレスを変更したりするたびに、全世界のコンピュータが持つ一覧表を更新して回ることなど、到底できません。この問題を解決するために生まれたのが、この章の後半で見ていくDNSと、その頂点に立つルートサーバー群です。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

まずは、IPアドレスという番号そのものを、機器に自動的に配り届ける仕組みから見ていきましょう。

10.1　DHCP

DHCP（Dynamic Host Configuration Protocol）は、ネットワークに接続した機器に対して、IPアドレスやデフォルトゲートウェイなどの設定情報を自動的に配布する仕組みです。

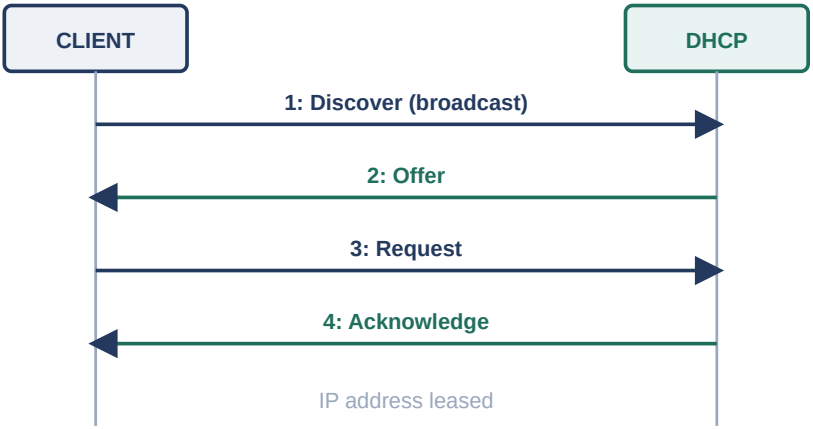
第4章で見てきたIPアドレスやデフォルトゲートウェイは、本来、機器ごとに手作業で設定する必要があります。しかし、社内に何百台もの機器があるとき、1台ずつ手作業でIPアドレスを設定して回るのは非現実的ですし、設定ミスによるIPアドレスの重複も起こりやすくなります。この問題を解決するのが、DHCPです。RFC 2131として標準化されています※1。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

DHCPが自動配布する主な情報

- IPアドレスとサブネットマスク（第4章）
- デフォルトゲートウェイのIPアドレス（第4章）
- DNSサーバーのIPアドレス（この章の後半で扱います）

DHCPによるIPアドレスの取得は、4段階のやり取りで行われ、それぞれの頭文字を取ってDORA（Discover, Offer, Request, Acknowledge）と呼ばれています。



DHCPのDORAプロセス

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

DORAプロセスの流れ

- Discover: クライアントが「DHCPサーバーはいませんか」とブロードキャストで問いかける
- Offer: DHCPサーバーが「このIPアドレスを使えます」と提案を返す
- Request: クライアントが「その提案のIPアドレスをください」と要求する
- Acknowledge: DHCPサーバーが割り当てを確定し、クライアントがそのIPアドレスを使い始める

配布されたIPアドレスには、多くの場合「リース期間」という有効期限が設定されており、期限が近づくとクライアントは同じDHCPサーバーに更新を依頼します。機器がネットワークから長期間離れるなどして更新されなければ、そのIPアドレスは他の機器に再利用できる状態に戻ります。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

10.2　豆知識：家庭のPPPoE（読み飛ばし可）

PPPoE（PPP over Ethernet）は、家庭のインターネット回線でよく使われる、イーサネット上でPPPというプロトコルを動かし、通信事業者との間で認証・接続を確立する仕組みです。

自宅のインターネット回線、特に光回線やDSL回線では、ルータの設定画面に「PPPoE接続」という項目や、プロバイダから渡された「ID」「パスワード」を入力する欄があることがあります。これは、電話回線の時代からあったPPP（Point-to-Point Protocol）という1対1の接続を確立するための仕組みを、イーサネット（第2章・第3章で見たLANの技術）の上で動かせるようにしたもので、RFC 2516として標準化されています※2。

家庭のルータは、この認証情報をもとにプロバイダ側の設備とPPPoEセッションを確立し、そのセッションを通じてグローバルIPアドレスなどの情報を受け取ります。自宅のルータの設定画面を開くと、この章で扱ったDHCP（ルータから

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

宅内の機器への配布）とPPPoE（ルータからプロバイダへの接続）という、2種類の異なる仕組みが同時に動いている様子を、実際に確認できることがあります。

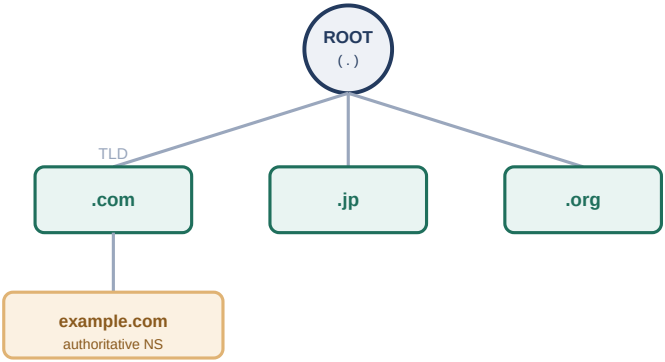
10.3　hostsファイルから外部参照、そしてルートDNSへ

組織の外にある無数のコンピュータの名前を、1枚の対応表で管理することは不可能なため、名前の管理を階層的に分担する仕組みとして、DNS（Domain Name System）が生まれました。

インターネットの黎明期には、接続されているすべてのコンピュータの名前とIPアドレスの対応関係を1つのファイル（`HOSTS.TXT`）にまとめ、それを定期的に配布するという運用がとられていました。しかし、接続されるコンピュータの数が増えるにつれて、この方法には無理が生じてきます。更新が反映されるまでの遅延、ファイルサイズの肥大化、そして何より、世界中の変更を1か所で管理すること自体の限界です。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

この課題を解決するために設計されたのが、DNS（Domain Name System）です。DNSは、名前の管理を「誰か1人がすべて把握する」のではなく、階層構造に分割し、それぞれの階層を別々の管理者が分担する、という発想に基づいています。全体的な概念や仕組みはRFC 1034※3、具体的なデータ形式や実装はRFC 1035※4として標準化されています。



DNSの階層構造: ルート → トップレベルドメイン → 個別ドメイン

この階層の頂点に立つのが、ルートサーバーです。ルートサーバーは、末端のドメイン名そのものを知っているわけではなく、「`.com`について知りたければ、こちらのサーバーに聞いてください」というように、次に問い合わせるべき先を

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

案内する役割を担っています。この階層を、ルート→トップレベルドメイン（.com、.jpなど）→個別のドメイン、とたどっていくことで、世界中のどの名前についても、最終的には正しい答えにたどり着けるようになっています。

10.4　DNSの仕組み

利用者の代わりに階層を順にたどって名前を解決してくれる「フルサービスリゾルバ」と、答えを一定期間覚えておくキャッシュの仕組みによって、DNSは日々の通信を裏で支えています。

わたしたちが普段、Webブラウザにドメイン名を入力したとき、実際には次のような流れで名前が解決されています。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

名前解決の流れ（概略）

- 手元の機器は、まず「フルサービスリゾルバ」（多くの場合、ISPやDHCPで指定されたDNSサーバー）に問い合わせる
- フルサービスリゾルバは、必要に応じてルートサーバー→トップレベルドメインのサーバー→個別ドメインの権威サーバー、と順に問い合わせを重ね（反復問い合わせ）、最終的な答えを手元の機器に返す
- 一度解決した結果は、一定時間（TTL、Time To Live）だけフルサービスリゾルバ側に記憶（キャッシュ）され、同じ問い合わせが来てもすぐに答えられるようにする（第7章のパケットのTTLとは名前が同じだけで別の仕組みです。こちらはDNSレコードごとにキャッシュを保持しておく時間を指します）

このキャッシュの仕組みのおかげで、世界中の利用者が同じ人気サイトへアクセスするたびに、いちいちルートサーバーまで問い合わせが飛ぶ、という事態が避けられています。また、DNSの問い合わせは、多くの場合、第7章で見たUDPの53番ポートを使って行われます。1回のやり取りで完結する軽量な問い合わせという性質上、コネクションを確立する手間のかかるTCPよりも、UDPのほうが向いているためで

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

す（大きな応答データを扱う場合など、一部の状況ではTCPも使われます）。

> クラウド環境では、この章で見たDHCPとDNSのどちらも、専用のマネージドサービスとして提供されているのが一般的です。仮想ネットワーク（VPC）ごとに「DHCPオプションセット」（AWSでの呼び方。Azure/GCPでは仮想ネットワークのDNS設定として同様の機能が提供されます）を指定すると、そのネットワーク内の機器に配布するDNSサーバーのアドレスやドメイン名をまとめて設定できますし、Amazon Route 53やAzure DNS、Cloud DNSといったマネージドDNSサービスを使えば、自前でDNSサーバーを構築・運用しなくても、権威DNSサーバーとしての機能を利用できます。オンプレミス環境とクラウド環境の両方に名前解決の仕組みがある場合には、双方を橋渡しするハイブリッドDNS（クラウド側のリゾルバ経由でオンプレミスのドメインも解決できるようにする構成など）も使われます。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

参考文献

- ※1: RFC 2131, *Dynamic Host Configuration Protocol* — https://www.rfc-editor.org/rfc/rfc2131.html
- ※2: RFC 2516, *A Method for Transmitting PPP Over Ethernet (PPPoE)* — https://www.rfc-editor.org/rfc/rfc2516.html
- ※3: RFC 1034, *Domain Names - Concepts and Facilities* — https://www.rfc-editor.org/rfc/rfc1034.html
- ※4: RFC 1035, *Domain Names - Implementation and Specification* — https://www.rfc-editor.org/rfc/rfc1035.html

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

第11章　アプリケーション層のサービス

11.1　HTTPとHTTPS

HTTPはWebページをやり取りするための取り決めで、HTTPSは、そのHTTPの通信をTLSという仕組みで暗号化したものです。

ここまでの章で見てきたIPアドレス・ポート番号・TCP/UDPは、いずれも「通信を届けるための仕組み」でした。この章で扱うのは、その届いた通信の中身として、実際にどんなやり取りが行われているか、というアプリケーション層（L7）のサービスです。

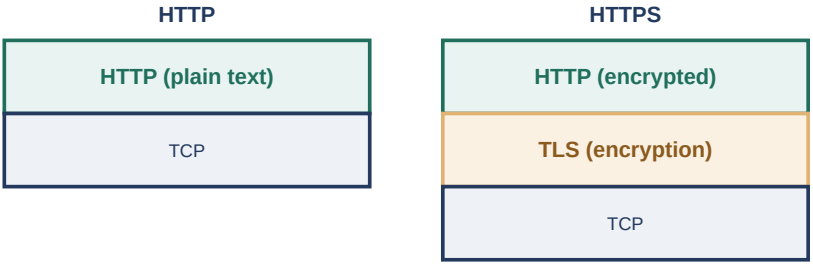
その代表格が、Webページの閲覧に使われるHTTP（HyperText Transfer Protocol）です。HTTPは、ブラウザ（クライアント）が「このページをください」というリクエストを送り、Webサーバーが該当するデータをレスポンスとして返す、という単純なリクエスト・レスポンス形式で成

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

り立っています。現行の仕様はRFC 9110としてまとめられています※1。

HTTPとHTTPSの違い

- HTTP: リクエスト・レスポンスの中身がそのまま（平文で）流れる
- HTTPS: TLS（Transport Layer Security）という暗号化の仕組みでHTTPの通信全体を包み、途中で盗み見られても内容が読めないようにする



HTTPSは、TCPとHTTPの間にTLSという暗号化の層を挟む

現行のTLSの仕様はRFC 9846としてまとめられています※2。銀行やショッピングサイトはもちろん、現在では大半

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

のWebサイトが、通信内容を保護するためにHTTPSを使うようになっています。

なお、近年では、HTTP/3という新しいバージョンも登場しています。これまでのHTTPはTCPの上で動いていましたが、HTTP/3は、QUICと呼ばれるUDPベースの新しいトランスポートの上で動作します（RFC 9114）※3。接続確立の速度や、パケット損失時の挙動の改善が主な狙いですが、輻輳制御などの詳細な仕組みは本書の範囲を超えるため、「そういう新しい方式がある」という程度にとどめておきます。

11.2　SSH

SSH（Secure Shell）は、離れた場所にある機器に安全にログインし、操作するための仕組みです。

ネットワーク機器やサーバーの多くは、直接その場で操作するのではなく、離れた場所からリモートで管理・操作されます。かつては、telnetという仕組みがよく使われていましたが、telnetはユーザ名やパスワード、操作内容がすべて平文で流れてしまうため、盗聴のリスクがありました。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

この問題を解決するために作られたのが、SSHです。通信内容を暗号化したうえで、リモートの機器にログインし、コマンドを実行できるようにします。RFC 4251としてプロトコルの全体構成が標準化されています※4。今日では、ルータやスイッチ、サーバーへのリモートログインの手段として、SSHが事実上の標準になっており、telnetは特別な事情がない限り使われなくなっています。

11.3　その他のサービス（メール、FTP、プロキシは概要のみ）

メールの送受信、ファイル転送、プロキシサーバーも、それぞれ専用のアプリケーション層のプロトコルによって成り立っています。

ここまで見てきたHTTP/HTTPS、SSHのほかにも、日常的に使われているアプリケーション層のサービスがいくつかあります。ここでは、深入りはせず、概要だけを押さえておきましょう。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

その他の主なサービス

- メール: メールを送信する際にはSMTP（Simple Mail Transfer Protocol、RFC 5321）※5が、受信したメールをメールサーバーから取り出す際にはPOP3やIMAPといったプロトコルが使われる
- FTP（File Transfer Protocol）: ファイルを転送するための古くからある仕組み。制御用とデータ転送用に別々の接続を使う独特な構造を持つ
- プロキシ: クライアントとサーバーの間に立ち、通信を代理で中継する機器やソフトウェア。クライアント側の代理をする「フォワードプロキシ」（社内からインターネットへのアクセスを一括管理する用途など）と、サーバー側の代理をする「リバースプロキシ」（複数のサーバーへの負荷分散や、TLSの終端など）がある

これらのサービスも、結局のところ、この章の前半で見たHTTPやSSHと同じように、決まったポート番号（メール送信なら25番や587番、FTPなら20番・21番など）を使い、決まった手順でやり取りを行う、という点では共通しています。第7章で見たポート番号とトランスポート層の知識があれば、初めて見るアプリケーション層のプロトコルに出会っても、「このプロトコルは、どのポート番号で、どんな手順

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

のやり取りをしているのか」という視点で読み解けるようになります。

> クラウド環境では、この章で見た仕組みの多くが、マネージドサービスとして提供されています。ロードバランサ（AWSのALB/NLBやAzureのApplication Gatewayなど）にTLSの終端を任せ、証明書の発行・更新もACM（AWS Certificate Manager）やAzure Key Vaultのようなサービスに委ねれば、各サーバーが個別に証明書を管理する手間を減らせます。SSHについても、踏み台サーバーを直接インターネットに公開する代わりに、Session Manager（AWS Systems Manager）やAzure Bastionのような仕組みを使い、SSHの鍵管理・ポート開放そのものを不要にする構成も一般的になっています。また、この章で見たリバースプロキシの発想は、クラウドのAPI Gateway（複数のバックエンドサービスへのルーティングや認証をまとめて引き受ける仕組み。AWS API GatewayやAzure API Managementなど）やCDN（コンテンツ配信ネットワーク）にも受け継がれています。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

参考文献

- ※1: RFC 9110, *HTTP Semantics* — https://www.rfc-editor.org/rfc/rfc9110.html
- ※2: RFC 9846, *The Transport Layer Security (TLS) Protocol Version 1.3* — https://www.rfc-editor.org/rfc/rfc9846.html
- ※3: RFC 9114, *HTTP/3* — https://www.rfc-editor.org/rfc/rfc9114.html
- ※4: RFC 4251, *The Secure Shell (SSH) Protocol Architecture* — https://www.rfc-editor.org/rfc/rfc4251.html
- ※5: RFC 5321, *Simple Mail Transfer Protocol* — https://www.rfc-editor.org/rfc/rfc5321.html

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

第12章　セキュリティと暗号化

12.1　情報セキュリティの3要素（CIA）

情報セキュリティは、機密性（Confidentiality）・完全性（Integrity）・可用性（Availability）という3つの要素の頭文字を取って、CIAと呼ばれる観点で整理されます。

ここまでの章では、通信を「届ける」ための仕組みを中心に見てきました。しかし、通信が正しく届くだけでは十分ではありません。届いた情報が正しい相手以外には見えないこと、途中で改ざんされていないこと、そして必要なときにきちんと使える状態であることも、同じくらい重要です。この3つの観点を整理したものが、CIAの3要素です。NIST（米国国立標準技術研究所）のFIPS 199でも、この3つの喪失をセキュリティ上のリスクとして定義しています※1。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

CIAの3要素

- 機密性（Confidentiality）: 許可された人だけが情報にアクセスできる状態を保つこと
- 完全性（Integrity）: 情報が不正に書き換えられたり、破壊されたりしていない状態を保つこと
- 可用性（Availability）: 必要なときに、情報やシステムに正しくアクセスできる状態を保つこと

この章で扱うファイアウォールやVPNは主に機密性を、暗号化は機密性と完全性の両方を支える技術です。可用性については、第9章で見た冗長化の仕組み（SPOFの排除、HSRP/VRRP、LAGなど）が中心的な役割を果たします。つまり、この本でここまで見てきた技術の多くは、実はすでにCIAのどれかを支えていた、と捉え直すこともできます。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

12.2　ファイアウォールとDMZ

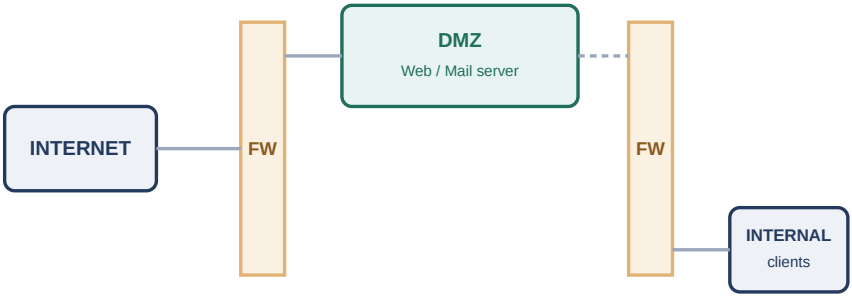
ファイアウォールは、ネットワークの境界で通信を検査し、許可されていない通信を遮断する機器・仕組みで、DMZは、外部に公開するサーバーを、社内ネットワークとは切り離した緩衝地帯に置く構成です。

第8章では、ACLという、ルールに基づいて通信を許可・拒否する仕組みを見ました。ファイアウォールは、このACL的な発想をさらに発展させ、通信の状態（ステートフルインスペクション）やアプリケーションの中身まで検査対象に含めた、境界防御の専用機器・機能です。NISTのSP 800-41 Rev. 1（2009年発行、後継の改訂版はまだ出ていません）では、ファイアウォールの種類や配置方針が整理されています※2。

企業のネットワークでは、外部（インターネット）に公開する必要があるサーバー（Webサーバーやメールサーバーなど）と、社内でしか使わない機器を、同じ扱いにするわけにはいきません。外部公開サーバーが仮に乗っ取られたとしても、そこから社内ネットワークの奥深くまで侵入されては困

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

るからです。そこで使われるのが、DMZ（Demilitarized Zone、非武装地帯）という構成です。



DMZ: 外部・DMZ・内部を、それぞれ別のファイアウォールルールで区切る（破線はDMZから内部への限定的なアクセスを示す）

DMZに置かれたサーバーは、インターネットからのアクセスは許可されますが、社内の内部ネットワークへ自由にアクセスすることは許可されません。逆に、内部ネットワークの機器からDMZへのアクセスは、必要な範囲でのみ許可されます。このように、外部・DMZ・内部という3つのゾーンをそれぞれ別のファイアウォールルールで区切ることで、1か所が突破されても、残りの区画への影響を最小限に抑えられるようにするのが、DMZという考え方です。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

12.3　暗号化はどこまで扱うか

暗号化には、送信者と受信者が同じ鍵を共有する共通鍵暗号と、公開鍵と秘密鍵の組を使う公開鍵暗号の、大きく2つの方式があります。

第11章では、HTTPSがTLSという仕組みでHTTP通信を暗号化していることを見ました。この章では、その暗号化そのものの考え方を、数式には立ち入らず、概念のレベルで押さえておきます。

暗号化の2つの方式

- 共通鍵暗号（Symmetric Key Cryptography）: 送信者と受信者が、事前に共有した同じ鍵を使って、暗号化と復号を行う方式。処理が高速な一方、鍵そのものを安全に相手へ届ける方法が課題になる
- 公開鍵暗号（Public Key Cryptography）: 公開してよい「公開鍵」と、本人だけが持つ「秘密鍵」という対になる鍵を使う方式。公開鍵で暗号化した内容は、対応する秘密鍵でしか復号できない

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

実際のHTTPS通信では、この2つの方式が組み合わされています。まず、公開鍵暗号を使って、通信のたびに使い捨てる共通鍵（セッション鍵）を安全に交換し、その後の実際のデータのやり取りは、処理が高速な共通鍵暗号で行う、という流れです。公開鍵暗号だけで大量のデータをやり取りするのは処理負荷が大きいため、「鍵交換には公開鍵暗号、本体の暗号化には共通鍵暗号」という役割分担が、多くの暗号化通信で採用されています。

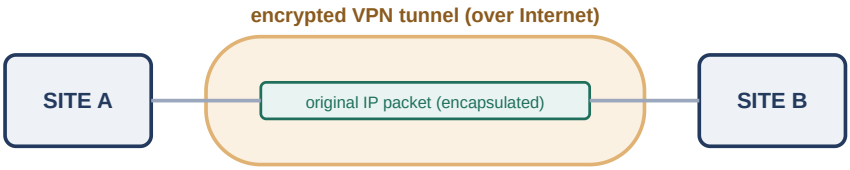
なお、第1章でOSI参照モデルのL6（プレゼンテーション層）が「文字コードや暗号化・圧縮など、データの表現形式を扱う」層として紹介されていたことを覚えているでしょうか。第1章で見たとおり、実際のインターネットの世界では、TLSはL6という独立した層としてではなく、HTTPSというアプリケーションの中に組み込まれる形で実装されています。ただし、「データの中身をどう表現し、どう保護するか」という関心事そのものは、まさにこのL6が担う役割にあたります。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

12.4　VPN

VPN（Virtual Private Network）は、インターネットのような公衆網の上に、暗号化によって保護された仮想的な専用回線を作り出す技術です。

支社と本社を専用回線で結ぶには、多くの場合、高額な回線契約が必要です。VPNは、すでに世界中に張り巡らされているインターネットの上に、暗号化された「トンネル」を作ることで、あたかも専用回線でつながっているかのような通信を、低コストで実現する技術です。



VPNトンネル: 元のパケットを暗号化してカプセル化し、公衆網の上を運ぶ

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

拠点間を結ぶVPN（IPsecなど）の多くは、IPパケットそのものを暗号化し、別のIPパケットの中に包み込む（カプセル化する）ことで実現されています。IPsecはRFC 4301として標準化されている、IP層でのセキュリティを提供する枠組みです※3。また、個々の利用者が外出先から社内システムへ安全に接続する用途では、第11章で見たTLSの仕組みをそのまま使うSSL-VPN（SSLはTLSの旧称で、名前だけがこの呼び方に残っています）もよく使われます。

> VPNクライアントの設定でよく登場する「スプリットトンネリング」は、すべての通信をVPN経由にするのではなく、社内システム宛ての通信だけをVPNトンネルに通し、それ以外のインターネット向け通信は通常の回線を直接使う、という設定です。VPN機器の負荷を減らせる一方、VPNを通らない通信は社内のセキュリティ対策の外側を通ることになるため、組織のセキュリティ方針に応じて使い分けが必要です。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

近年では、拠点やVPNを「境界」として信頼するのではなく、利用者・端末・通信そのものを毎回検証する、ZTNA（Zero Trust Network Access）という考え方への移行が進んでいます。NISTはSP 800-207として、このゼロトラストアーキテクチャの考え方を整理しています※4。この章で見たDMZやVPNが「境界の内側は信頼する」という前提に立つのに対し、ZTNAは「社内ネットワークの内側だから安全、とは考えない」という前提に立つ点が大きな違いです。クラウドサービスの利用が広がり、守るべき対象が単一の社内ネットワークの中に収まらなくなったことが、この移行の背景にあります。

参考文献

- ※1: NIST FIPS 199, *Standards for Security Categorization of Federal Information and Information Systems* — https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.199.pdf

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

- ※2: NIST SP 800-41 Rev. 1, *Guidelines on Firewalls and Firewall Policy* — https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-41r1.pdf
- ※3: RFC 4301, *Security Architecture for the Internet Protocol* — https://www.rfc-editor.org/rfc/rfc4301.html
- ※4: NIST SP 800-207, *Zero Trust Architecture* — https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207.pdf

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

終章　学び直しの総括

ここまでの振り返り

序章で1台のコンピュータから始まった旅は、機器同士をつなぐ物理層から、組織を越えて世界とつながるアプリケーション層まで、積み重なる階層をひとつずつたどる旅でした。

序章では、ハードウェアとソフトウェアの技術進歩、コストの最適化、複雑さと単純さの間を行き来しながら、ネットワークの歴史が形を変えてきた道のりを振り返りました。ここでは、その歴史の中で登場した技術要素が、この本の中でどう積み重なっていったかを、章をまたいで見渡してみましょう。

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

積み上げてきた地図

- 層という共通言語: 通信を7つの層に分けて捉える見方（第1章のOSI参照モデル）
- 物理層: ケーブルや電波で、0と1を届ける（第2章）
- データリンク層: 同じ配線の中で、MACアドレス宛てに届ける（第3章）、それを論理的に分割する（第6章のVLAN）
- ネットワーク層: 違うネットワーク同士をIPアドレスで結ぶ（第4章）、経路を選ぶ（第5章のルーティング）
- トランスポート層: どのアプリケーション宛てかを区別し、必要なら信頼性を保証する（第7章）
- アプリケーション層: 実際のサービスとしてやり取りする（第11章）
- 運用を支える技術: 通信を選別し守る（第8章のACL/NAT、第12章のセキュリティ）、止まらないようにする（第9章の冗長化）、自動化する（第10章のDHCP/DNS）

この本は、単に技術要素を年代順・階層順に並べただけではなく、各章の中に伏線と回収を仕込んできました。第3章の「ブロードキャストドメインを分割できるのは、L3（ネットワーク層）で動作するルータだけ」という断言が、第6章のVLANによってどんでん返しを迎えたように、ネットワー

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

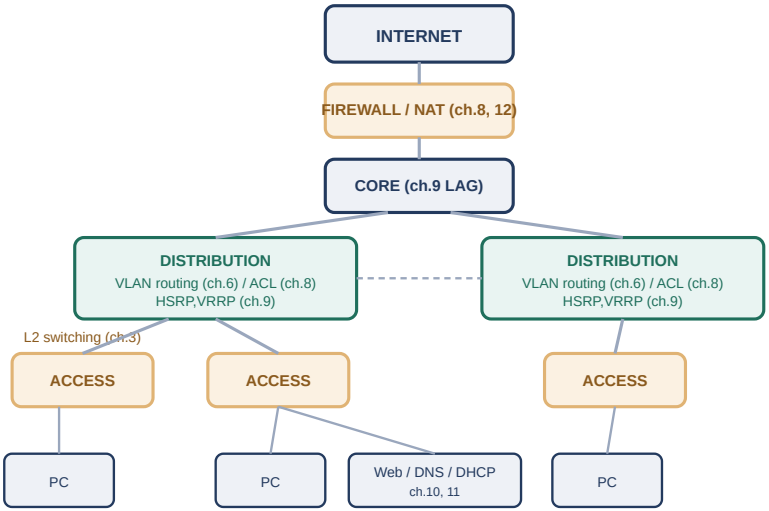
クの世界では、ある層で「これしかできない」と思われていたことが、別の層の技術によって乗り越えられる、ということがよく起こります。この本を読み終えたいま、みなさんの頭の中には、個々の技術の知識だけでなく、それらがどうつながり合っているかという地図ができあがっているはずです。

ワーク1　重ね合わせワーク

1つの小さな社内ネットワークの構成図を題材に、序章から積み上げてきた各層の登場人物を、1枚の図の上に重ね合わせて見直してみましょう。

次の図は、第9章で見た3層ネットワークアーキテクチャをベースにした、ごく小規模な社内ネットワークの例です。この図の中に、この本で学んできた要素がどこに現れているかを、実際に指差しながら確認してみてください。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode



この本で学んだ要素を、1つのネットワーク構成図に重ね合わせる

- PCとアクセススイッチをつなぐケーブルは第2章（物理層）、そのフレームのやり取りは第3章（データリンク層）
- アクセススイッチ上でのVLANの区切りは第6章、ディストリビューション層でのVLAN間ルーティングは第6章と第4章・第5章の組み合わせ
- ディストリビューション層のHSRP/VRRPによる冗長化は第9章、同じくディストリビューション層でのACL

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

によるポリシー適用と、境界でのNATは、いずれも第8章

- インターネットとの通信で使われるポート番号・TCP/UDPは第7章、DHCPによるIPアドレスの自動配布とDNSによる名前解決は第10章
- Webサーバーへのアクセスやそこで動くHTTP/HTTPSは第11章、通信を暗号化するVPNやファイアウォールで内と外を区切るという考え方は第12章

こうして見ると、たった1枚の小さな構成図の中に、この本のほぼすべての章が顔を出していることに気づくはずです。実際のネットワーク構成図を眺めるときも、同じように「これはどの層の話か」「どの章で学んだ技術か」を意識してみると、複雑に見える図も、いくつかの層に分解して理解できるようになります。

ワーク2　設計思考ワーク

学んだ要素を組み合わせて、簡単な条件に沿ったネットワーク構成を、自分の手で設計してみましょう。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

次のような条件を持つ、小さな会社のネットワークを設計するとしたら、みなさんならどう構成するでしょうか。

お題の条件（例）

- 部署は「営業」「開発」「総務」の3つがあり、それぞれ十数台のPCを使う
- 開発部門は、社外に公開するWebサーバーを1台、社内ネットワークとは別の区画で運用したい
- インターネット回線が1本しかなく、それが落ちると全社の業務が止まってしまうことが課題になっている
- 開発部門のPCから営業部門のPCへは、業務上アクセスできる必要はない

この条件だけを見ても、この本で学んだ技術の多くが、そのまま解決の糸口になることに気づくはずです。部署ごとの分離には第6章のVLANが、Webサーバーの隔離には第12章のDMZが、インターネット回線の単一障害点という課題には第9章の冗長化の考え方が、それぞれ対応します。正解は1つではありません。実際に紙やホワイトボードに構成図を描き、「なぜその構成にしたのか」を自分の言葉で説明できるところまで、一度試してみることをおすすめします。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

ワーク3　アウトプットの推奨

学んだ内容は、誰かに説明したり、文章として書き出したりすることで、初めて本当に自分のものになります。

この本を読み終えた後、ぜひ試してほしいのが、学んだ内容をアウトプットすることです。同僚に「VLANって何のためにあるの」と聞かれたときに、この本で見てきた「1台の機械の中に、複数の独立したネットワークがあるかのように見せかける」という発想を、自分の言葉で説明できるでしょうか。あるいは、ブログや社内Wikiに、この本で学んだ内容を自分なりに整理し直して書き出してみるのもよい方法です。

人に説明しようとすると、自分がどこを本当に理解していて、どこがまだ曖昧なままなのかが、驚くほどはっきりと見えてきます。そして、その曖昧さに気づいたときこそ、該当する章に戻って読み返す、よいタイミングです。ネットワークの学び直しは、この本を読み終えた瞬間に終わるのではなく、実際に手を動かし、人に説明し、また分からないことに

© Takashi Kouno • CC BY-NC-ND 4.0 • creativecommons.org/licenses/by-nc-nd/4.0/legalcode

ぶつかる、というサイクルの中で、少しずつ続いていくものです。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

ライセンスについて

本書（Web版・PDF版を含む本文・図版等のコンテンツ）は、**Creative Commons 表示 - 非営利 - 改変禁止 4.0 国際 (CC BY-NC-ND 4.0)** ライセンスの下で提供されています。

- 正式なライセンス条文（英語）：https://creativecommons.org/licenses/by-nc-nd/4.0/legalcode
- 人間可読な要約（Deed）：https://creativecommons.org/licenses/by-nc-nd/4.0/

このライセンスが意味すること

CC BY-NC-ND 4.0 は、次の3つの条件の組み合わせです。

- **表示（BY）** 著作者名（河野 崇 / Takashi Kouno）を表示し、ライセンスへのリンクを示し、変更を加えた場合はその旨を明示してください。

- **非営利（NC）** 商業的な利益や金銭的対価を主目的とした利用はできません。個人のブログでの紹介や、学校の授業での非営利利用など、営利を目的としない利用は問題ありません。
- **改変禁止（ND）** 原文を改変・翻案（要約、翻訳、部分的な書き換えなど）した二次的著作物を作成し、それを共有することはできません。原文をそのままの形で共有することは可能です。

つまり、「**著作者を明示し、非営利目的であれば、本書をそのままの形で自由に共有してよい**」というのが、このライセンスの基本的な考え方です。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

具体例

行為	可否
個人のブログやSNSで本書のリンクを紹介する	可能
学校の授業で非営利の教材として配布する（原文のまま）	可能
本書をそのままの形でPDF等で再配布する	可能（著作者表示・ライセンス表示が必要）
本書の文章を要約・翻訳して別サイトに公開する	不可（改変にあたるため）
本書を書籍化・有料コンテンツ化して販売する	不可（非営利条件に反するため）
図版の一部だけを切り出して別の記事に転載する	不可（改変とみなされる可能性が高い）

この表示は第三者による無断利用を制限するものであり、著作権者自身が本書を利用すること（販売や研修での利用等）を制限するものではありません。また、個別の相手への

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

無償・優遇条件での利用許諾は、このライセンス文言とは別に、著作権者の判断で行うことがあります。

利用にあたって迷った場合は、リポジトリの Issue にてご相談ください。

© Takashi Kouno ・ CC BY-NC-ND 4.0 ・ creativecommons.org/licenses/by-nc-nd/4.0/legalcode

奥付

書名　ネットワーク基礎再入門

著者　河野 崇（Takashi Kouno）

発行　（自費出版 / Web 公開）

初版発行日　2026年8月24日

リポジトリ　https://github.com/shimesaba-type0/network-intro-text

本書（本文・図版等のコンテンツ）は、 Creative Commons 表示 - 非営利 - 改変禁止 4.0 国際（CC BY-NC-ND 4.0）ライセンスの下で提供されています。

- 著作者名の表示（Takashi Kouno）とライセンス表示を行うこと
- 非営利目的での利用に限ること
- 原文を改変せず、そのままの形で共有すること

を条件に、本書の複製・共有を自由に行うことができます。詳しくは「ライセンスについて」の章、および正式なライセンス条文

（https://creativecommons.org/licenses/by-nc-nd/4.0/legalcode）を参照してください。

© Takashi Kouno

TAKASHI WORKS